

Bachelor-Arbeit

Deep Learning-gestützte
Ausrichtung von räumlichen
Entitäten für den 5D-Druck

Lars Naumann

9. Februar 2026

Gutachter:

PD Dr. Frank Weichert

Prof. Dr. Heinrich Müller

Lehrstuhl Informatik VII
Computergraphik
TU Dortmund

Inhaltsverzeichnis

Mathematische Notation	e
1 Einleitung	1
1.1 Motivation und Hintergrund	1
1.2 Aufbau der Arbeit	2
2 Topologieaspekte in der Additive Fertigung	5
2.1 3d-Druck	5
2.2 Überhang-Problem	6
2.3 Fertigungs-Geschwindigkeit	7
2.4 Bauteiloptimierung	9
2.5 5d-Druck	11
3 Flächensuche in Bauteilen	15
3.1 Kriterien für geeignete Flächen	15
3.2 Clustering nach Normalvektoren in Bauteilen	17
3.2.1 Normalvektoren	17
3.2.2 Hierarchisches Clustering	18
3.3 Separation von parallelen Flächen	21
4 Punktwolkenverarbeitung zur Bauteilausrichtung	27
4.1 Ziel der Punktwolkenverarbeitung	27
4.2 Rotationserstellung für Flächen	28
4.2.1 Maschinelles Lernen	28
4.2.2 Merkmalsextraktion aus Punktwolke	30
4.2.3 Regression auf Merkmalsvektor	34
4.3 Konstruktion der Rotationsmatrix	35
5 Evaluation	41
5.1 Versuchsaufbau	41
5.2 Ausrichtung der Bauteile	42

5.3	Verlässlichkeit der Ausrichtung	43
5.3.1	Effekt von Rauschen auf den Daten	44
5.3.2	Effekt von schlechter Flächensuche	45
5.4	Generalisierbarkeit	47
6	Zusammenfassung und Ausblick	51
6.1	Zusammenfassung	51
6.2	Ausblick	53
	Abbildungsverzeichnis	55
	Algorithmenverzeichnis	57
	Literatur	59

Mathematische Notation

Notation	Bedeutung
$\mathbb{N}$	Menge der natürlichen Zahlen $1, 2, 3, \dots$
$\mathbb{R}$	Menge der reellen Zahlen
$\mathbb{R}^d$	d -dimensionaler Raum
$\mathcal{M} = \{m_1, \dots, m_N\}$	ungeordnete Menge $\mathcal{M}$ von N Elementen m_i
$\mathcal{M} = \langle m_1, \dots, m_N \rangle$	geordnete Menge $\mathcal{M}$ von N Elementen m_i
$\mathbf{v}$	Vektor $\mathbf{v} = (v_1, \dots, v_n)^T$ mit N Elementen v_i
$v_i^{(j)}$	i -tes Element des j -ten Vektors
$\mathbf{A}$	Matrix $\mathbf{A}$ mit Einträgen $a_{i,j}$
$G = (V, E)$	Graph G mit Knotenmenge V und Kantenmenge E

1 Einleitung

1.1 Motivation und Hintergrund

Die additive Fertigung ist ein Feld mit viel Potenzial. Es ist eine Technologie, die in der Fertigung immer wichtiger wird. Schnelle Produktion von Prototypen und Ersatzteilen sind nur einige der Anwendungsbereiche von dem 3d-Druck [Fas12]. Das Feld des 3d Drucks hat im Jahr 2025 einem Wachstum von 9,1 % auf 21,9 Milliarden US-Dollar. Die Industrie zeigt seit 2009 konstantes Wachstum [WW25]. Allerdings gibt es noch Probleme, zum Beispiel mit der strukturellen Integrität von Bauteilen oder der Materialverschwendung durch Stützstrukturen. Der 5d-Druck biete eine Möglichkeit beide Probleme zu adressieren, indem das Bauteil während des Drucks gedreht wird. Dadurch ist es im 5d-Druck möglich Ausrichtungen für einzelnen Flächen eines Bauteils zu wählen, indem die Plattform auf der das Bauteil platziert wird, sich in zwei Achsen kippen kann. Um eine optimale Ausrichtung zu finden, gibt es bis jetzt noch keine weit anerkannte automatische Lösung. Eine Möglichkeit ist die Verwendung von neuronalen Netzen, um eine optimale Ausrichtung zu schätzen. Die Abbildung 1.1 zeigt einen 5d-Drucker. Es lässt sich erkennen, dass die Plattform gekippt ist.

Neurale Netze sind gut geeignet, Muster in Daten zu erkennen. Die Idee ist es die Informations-Verarbeitungsfähigkeit des menschlichen Gehirns auf einem Computer zu simulieren [Wan03]. Das Gebiet der künstlichen Neuralen Netze hat sich bereits in vielen Anwendungsfällen als verlässlich erwiesen. In der Bildverarbeitung wird es zum Beispiel verwendet, um Objekte in Bildern zu erkennen [KM21].

Es ist daher naheliegend, dass sie auch verwendet werden können, um Muster in einer Menge an Raumpunkten zu erkennen. Eines Neutrales Netz kann gut über Daten verallgemeinern und auch für unbekannte komplexe Daten eine gute Lösung finden, wo klassische Algorithmen versagen könnten. Auch für die Ausrichtung von Bauteilen haben sich neurale Netze bereits als gute Lösung erwiesen [Zel+25]. Es wurde mit der PointNet Architektur [Qi+17a] gezeigt, dass Neurale Netze in der

¹<https://i.ytimg.com/vi/B9sdrezl6AU/maxresdefault.jpg>



Abbildung 1.1: Beispiel für einen 5d-Drucker¹

Lage seinen können eine Ausrichtung für einfache geometrische Strukturen zu finden. Wenn man diese Idee weiterentwickelt könnte sie genutzt werden, um Ausrichtungen für die Flächen eines Bauteils im 5d-Druck zu bestimmen. Zu diesem Zweck ist es notwendig lokale Merkmale zu erkennen, was die Nutzung von PointNet++ erfordert [Qi+17b]. Es ist im Gegensatz zu Pointnet in der Lage lokale Merkmale zu erkennen. Das Ziel der Arbeit ist es zu bewerten, ob eine Teilausrichtung, basierend auf einem Neuralen Netz, eine praktikable Möglichkeit der Teilausrichtung ist. Eine geringe Abweichung von der vorher berechneten optimalen Ausrichtung ist zu erreichen. Eine möglichst geringe Bewegung des Druckers ist ebenfalls wünschenswert.

1.2 Aufbau der Arbeit

In dieser Arbeit wird versucht das Problem der Ausrichtung von Bauteilen im 5d-Druck durch die Verwendung von tiefen neuronalen Netzen zu lösen. Als Datensatz werden künstliche generierte Objekte verwendet. Begonnen wird mit der Aufteilung eines Bauteils zu in sich geschlossenen Flächen, wofür mehrere klassische Clustering verfahren genutzt werden. Anschließend wird ein Neutrales Netz trainiert, dass die Flächen eines Bauteils auf Rotationen abbildet, die das Bauteil optimal für den 5d-Druck ausrichten. Für das Neurale Netz wird eine angepasste Version von einem Bereits existierenden neuronalen Netz für die Merkmalsextraktion verwendet [Qi+17b]. Danach wird die Leistung des neuronalen Netzes evaluiert, anhand der Ausrichtung auf unterschiedlichen Objekten und der Widerstandsfähigkeit gegen Störungen. Es

wird ebenfalls auf nicht künstlich generierten Daten getestet.

Zuletzt wird diskutiert, ob die Ergebnisse darauf schließen lassen, ob Neurale Netze eine Praxisrelevante Ausrichtungsmethode darstellen.

2 Topologieaspekte in der Additive Fertigung

2.1 3d-Druck

Der 3d-Druck ist eine Methode der Fertigung. Es wird auch additive Fertigung genannt, da das Bauteil durch das Hinzufügen von Material Schicht für Schicht erstellt wird [Sta15]. Um ein Bauteil 3d-Drucken zu können, wird das Bauteil im Computer in mehrere Schichten aufgeteilt. Diese Schichten werden dann nacheinander gedruckt, bis das Bauteil fertig ist. Es gibt mehrere Typen von 3d-Druck. Die Typen beinhalten: Binding Jetting, Directed Energy Deposition, Material Extrusion, Material Jetting, Powder Bed Fusion und noch mehrere weitere. Alle Methoden tragen Material auf. Die am weitesten verbreitete Methode ist Materials Extrusion, bei der meistens Plastik erhitzt wird und dann das geschmolzene Plastik aufgetragen wird. Die Materialien die genutzt werden können sind aber mehr als Plastik und beinhalten unter anderem noch Metall, Polymere, Komposit Materialien und noch mehr. Metalle werden in der Luft und Raumfahrt verwendet, da sie sehr gute physische Eigenschaften haben. Das benutzte Metall kommt auf den genauen Anwendungsbereich an. Plastik hat den Vorteil, dass es leicht zu erhitzen und billig ist. Es wird unter anderem für die Produktion von Prototypen benutzt, kann aber auch für fertige Produkte verwendet werden. Komposit Materialien, wie Glasfaser, dagegen sind anpassungsfähig und leicht. Sie werden ebenfalls in der Luft und Raumfahrt eingesetzt. Der 3d-Druck ist eine Technologie mit vielen Anwendungsmethoden und Bereichen [SLR19].

In der Abbildung 2.1 ist eine schematische Darstellung eines 3d-Druckers zu sehen. Dabei ist oben die Düse, welche das Material erst schmilzt und dann aufträgt. Die Düse lässt sich in x und y Richtung bewegen und kann so jede beliebige Position auf der Plattform erreichen. Unten kann man die Plattform sehen, auf der das Bauteil gedruckt wird. Sie kann sich nach unten bewegen, damit Platz für die nächste Schicht

¹<https://www.linux-magazin.de/wp-content/uploads/2020/11/b01.jpg>

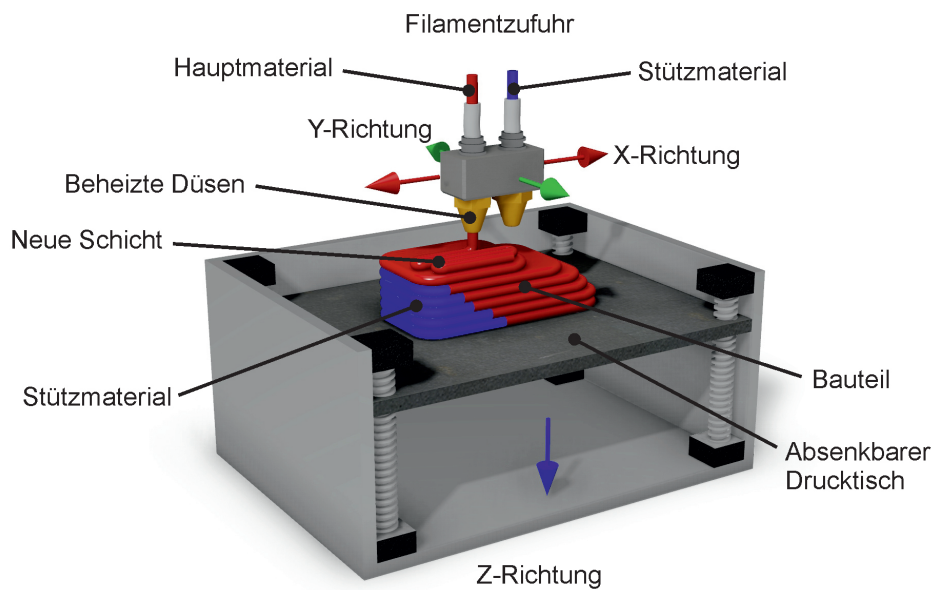


Abbildung 2.1: Schematische Darstellung eines 3d-Druckers¹

geschaffen wird. Die Funktion des Stützmaterials und der Stützsichten wird in Abschnitt 2.2 erklärt. Der in Abbildung 2.1 zusehende 3d Drucker ist ein Beispiel für den Aufbau eines 3d-Druckers. Es gibt 3d Drucker, die anders funktionieren, aber das Grundprinzip ist bei allen gleich. Alle haben eine Düse, die das Material ausgibt und diese Düse bewegt sich, um Schichten abzuarbeiten.

Bevor ein Objekt gedruckt werden kann, muss es in Schichten aufgeteilt werden. Das wird vor dem Drucken von Software erledigt. Für jede erstellte Schicht wird im Vorhinein ein Weg berechnet, mit dem die Düse die Schicht abfährt. Diesen Vorgang nennt man Slicing (de. In Scheiben Schneiden”).

2.2 Überhang-Problem

Im 3d-Druck werden Bauteile traditionell Schicht für Schicht gebaut. Das Bauteil wird vor dem Druck in dünne horizontale Schichten zerlegt. Jede Schicht wird dann nacheinander gedruckt. Dabei kann es zu Problemen kommen, wenn eine Schicht über eine vorherige Schicht hinausragt, ohne dass darunter Material ist. Dies wird als Überhang-Problem bezeichnet. Um diesem Problem entgegenzuwirken werden Stützstrukturen eingesetzt.

In der Abbildung 2.2 ist ein Bauteil zu sehen, das Stützstrukturen benötigt. Die Ab-

²<https://i.stack.imgur.com/cIDB3.png>

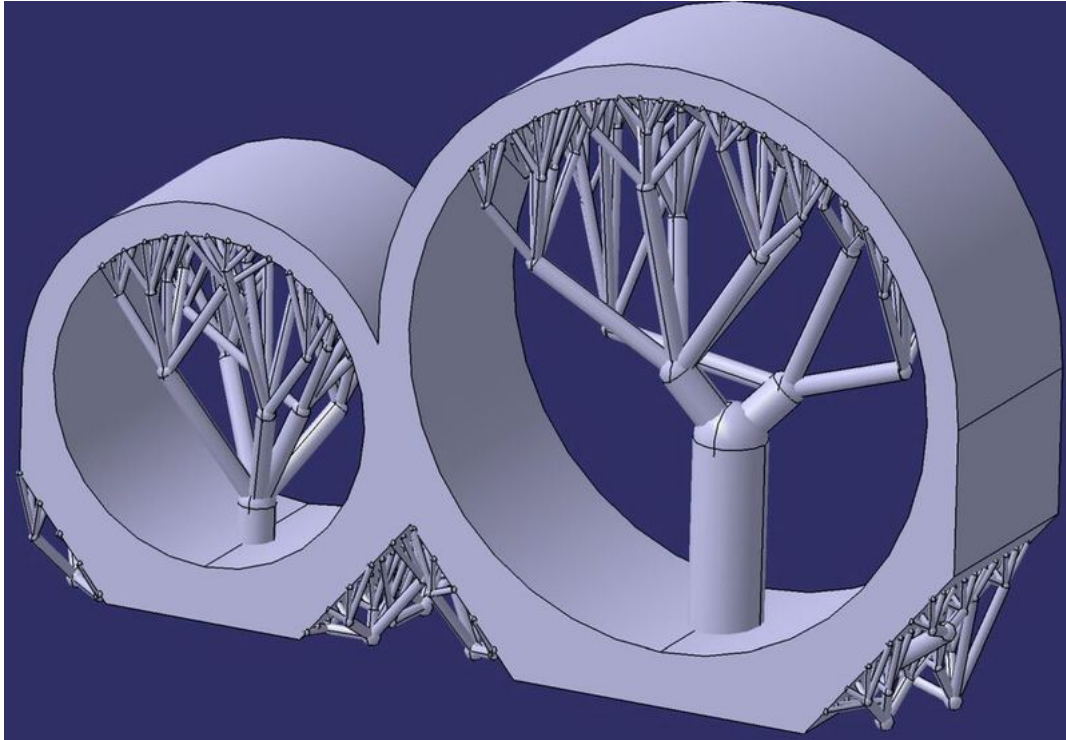


Abbildung 2.2: Vorschau eines Bauteils mit Stützstrukturen²

Abbildung 2.2 zeigt zwei Ringe, die sich an einer Stelle berühren. Sie sind das Bauteil, das gedruckt werden soll. Innerhalb beider Ringe sind Säulen, die die Decke der Ringe stützen. Das sind die, vor dem Druck hinzugefügten, Stützstrukturen. Stützstrukturen erhöhen den Materialaufwand, wie man in Abbildung 2.2 sehen kann. Die Strukturen werden nach dem Druck entfernt, was das zusätzlich benötigte Material zu Müll macht. Durch das Hinzufügen von Stützstrukturen verlängert sich auch die Druckzeit. Dieses Problem kann durch die richtige Ausrichtung des Bauteils minimiert werden, aber bei komplexen Bauteilen ist es unwahrscheinlich, dass eine Ausrichtung gefunden wird, die keine oder wenige Stützstrukturen benötigt. Stützstrukturen sind nicht der einzige Weg das Überhangsproblem zu bekämpfen. Eine Anpassung der Druckgeschwindigkeit und der Materialtemperatur können die Notwendigkeit von Stützstrukturen reduzieren. Dabei ist ein Überhang von 40 Grad ohne Stützstrukturen oder Verlust an Qualität möglich [Jia+18].

2.3 Fertigungs-Geschwindigkeit

Wie bei jeder Art der Fertigung ist die Produktionsgeschwindigkeit ein wichtiger Faktor. In der additiven Fertigung wird die Produktionsgeschwindigkeit durch ver-

schiedene Faktoren beeinflusst. Eine dieser Faktoren wurde in Abschnitt 2.2 beschrieben. Das Hinzufügen von Stützstrukturen erhöht die Druckzeit, da mehr Material gedruckt werden muss. Das ist ein weiterer Grund warum es wichtig ist, Stützstrukturen zu minimieren. Dementsprechend ist die Ausrichtung eines Bauteils wichtig, da eine gute Ausrichtung die benötigten Stützstrukturen minimieren kann.

Der offensichtlichste Faktor ist die Größe des Bauteils und die Druckgeschwindigkeit des Druckers. Ein größeres Bauteil benötigt mehr Zeit, wenn es mit der gleichen Geschwindigkeit gedruckt wird. Die Druckgeschwindigkeit ist abhängig vom Drucker. Unabhängig von dem Drucker gibt es physikalische Grenzen für die Druckgeschwindigkeit. Wenn eine bestimmte Geschwindigkeit überschritten wird, wird es zu Problemen mit der Genauigkeit und Qualität des Drucks kommen. Die Geschwindigkeit, mit der eine Düse Material ausgeben kann, ist ebenfalls ein limitierender Faktor.

Vor- und Nachbearbeitung ist ebenfalls ein Faktor in der Produktionsgeschwindigkeit. Die Vorverarbeitung oder Planungszeit ist direkt von der Komplexität eines Bauteils abhängig. Die Vorverarbeitung behandelt alles, was benötigt wird, um für ein 3d Modell eines Bauteils einen ausführbaren Bauplan für den Drucker zu erstellen. Einen Druck zu planen lässt sich in vier Phasen unterteilen: Die optimale Ausrichtung finden, das Bauteil in Scheiben aufteilen (eng. Slicing), notwendige Stützstrukturen hinzufügen und den Pfad für die Düse des Druckers erstellen [OP16].

Die Ausrichtung des Bauteils muss mehrere Aspekte berücksichtigen. Die wichtigsten sind das Minieren der Stützstrukturen und das Verbessern der Oberflächenqualität. Auswirkungen der Ausrichtung wird in Abschnitt 2.2 erklärt. Stützstrukturen erhöhen den Materialaufwand und die Druckzeit, da sie mit dem Rest des Bauteils gedruckt werden. Nach dem Druck werden sie von dem fertigen Bauteil entfernt und sind Müll, daher ist so wenig Zeit wie möglich mit Stützstrukturen zu verbringen häufig eine hohe Priorität [OP16].

In der Abbildung 2.3 ist der Unterschied, den die Höhe der Schichten auf die Qualität des Bauteils hat, zu sehen. Auf der linken Seite der Abbildung sind die Schichten dicker als auf der rechten Seite, auf beiden Seiten wird versucht dieselbe Kurve darzustellen. Es ist zu erkennen, dass die dünneren Schichten auf der rechten Seite der Abbildung die Kurve besser darstellen, als die dickeren auf der linken Seite. Umso dünner die Schicht desto genauer wird das Bauteil gedruckt. Wenn die Dicke der Schichten verringert wird, werden allerdings mehr Schichten benötigt, was die Strecke, die die Düse abarbeiten muss, erhöht. Dadurch verlängert sich die Druckzeit.

³https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcS6yH0djlG06mhqwt_JJ22BE7VaI-0V4U4SWg&w=1000&h=1000&from=webp

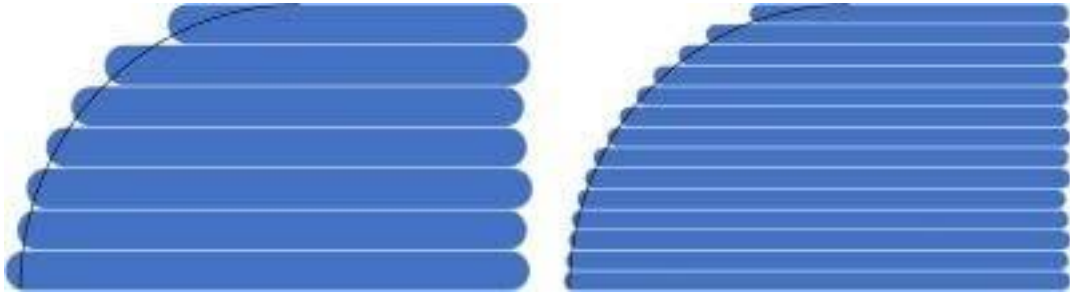


Abbildung 2.3: Darstellung der Unterschiede im Bauteil abhängig von der Höhe der Schichten³

In den meisten Fällen sind alle Schichten gleich dick, auch wenn es keine optimale Lösung ist. Diese Vereinfachung sorgt häufig dafür, dass ein gedrucktes Bauteil keine guten Übergänge zwischen den Schichten hat, was ein treppenartiges Aussehen zur Folge hat [OP16].

Der Pfad, den die Düse über eine Schicht nimmt, kann optimiert werden. Ein Düsenpfad (engl. Toolpath“) beeinflusst die Qualität und Geschwindigkeit stark [Zhu+24]. Ein längerer Pfad, den die Düse abarbeiten muss, resultiert in einer längeren Druckzeit.

In der Nachbearbeitung wird versucht Unebenheiten und Fehler auf der Oberfläche zu korrigieren. Die Oberfläche wird per Hand geglättet, der Vorgang ist häufig sehr zeitintensiv [OP16].

2.4 Bauteiloptimierung

Ein Objekt, welches gedruckt werden soll, kann verändert werden, um Material, Druckzeit und Gewicht einzusparen. Um Material zu sparen wird im Inneren eines Bauteils Material weggelassen. Das Weglassen von Material im Inneren eines Bauteils verändert die strukturellen Eigenschaften des Bauteils, dementsprechend ist es wichtig die Optimierung des Bauteils dem Anwendungsfall anzupassen. Die Berechnungen von Optimierungen ist sehr rechenintensiv, besonders bei komplexen Topologien [Fen+18].

Das generelle Vorgehen ist es ein Objekt mit wiederholbaren geometrischen Formen, wie Hexagone oder Quadrate, zu füllen. In der Abbildung 2.4 ist auf der linken Seite ein Bauteil mit gleich großen Quadrate gefüllt worden. Die schwarzen Kanten sind die Teile des Bauteils, die gedruckt werden. Dazwischen kann man Freiräume sehen.

⁴https://media.springernature.com/lw685/springer-static/image/art%3A10.1186%2Fs42492-018-0004-3/MediaObjects/42492_2018_4_Fig9_HTML.png?as=webp

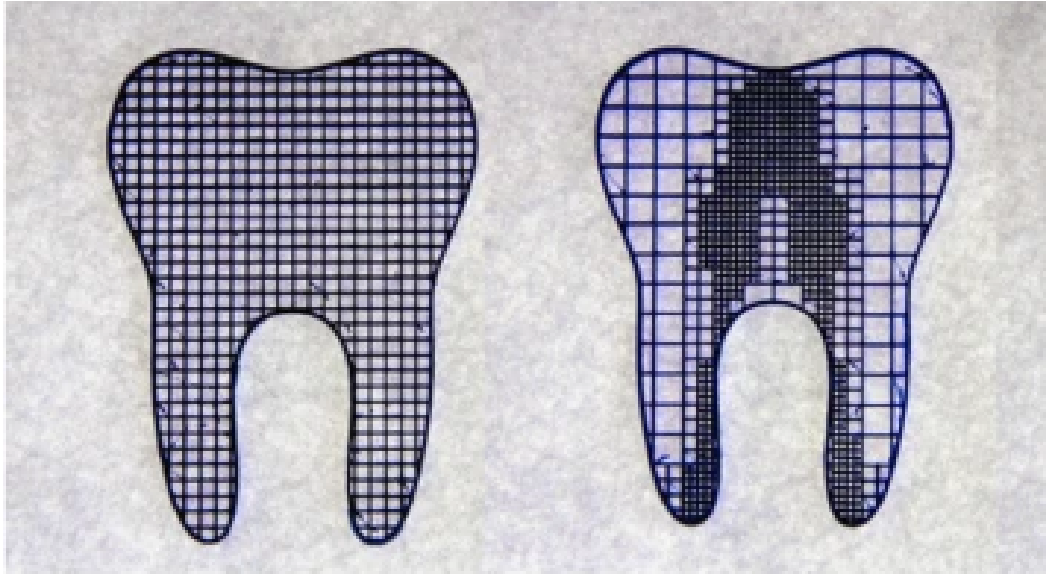


Abbildung 2.4: Darstellung von durch Formoptimierung aufgeteiltem Bauteil (links) und verbesserte Aufteilung durch Formoptimierung (rechts)⁴

In dem Bauteil rechtes in der Abbildung 2.4 ist dasselbe Bauteil, das auch links zu sehen ist mit unterschiedlich dichten Quadraten gefüllt. Eine geringere Größe der Quadrate sorgt für eine höhere Dichte an bestimmten Stellen. Die strukturelle Stärke des Bauteils ist proportional zu der Dichte der Füllung. Um die strukturelle Stärke an Bereichen mit höherer Belastung zu verstärken, kann die Dichte in benötigten Bereichen erhöht werden. In dem Zentrum des rechten Bauteils in Abbildung 2.4, wo die meiste Belastung zu erwarten ist, ist die Dichte des Gittermusters erhöht. An allen anderen Stellen ist es geringer, als im Vergleich zu dem linken Bauteil.

Die Form eines Bauteiles selber kann optimiert werden, um bestimmte Kriterien zu erfüllen. Dieser Vorgang wird Topologieoptimierung genannt. Es wird eine Struktur gesucht, die eine Zielfunktion minimiert und dabei die Einschränkungen im Volumen und andere Einschränkungen nicht überschreitet [SM13]. Die Idee ist es, die Form eines Bauteils zu verändern, um bestimmte Ziele, wie Material oder Gewichtseinsparungen, zu erreichen, ohne vom Anwendungsfall abhängige Anforderungen zu verletzen. Dafür kann Material entfernt oder hinzugefügt werden. Im Gegensatz zu dem Weglassen von Material im Inneren des Bauteils geht es bei der Topologieoptimierung die Form des Bauteils selber zu optimieren. Es lässt sich zum Beispiel an Löchern auf dünnen Flächen erkennen. Die Topologieoptimierung verändert das Aussehen eines Bauteils, bei vielen Arten von Bauteilen kann es dementsprechend nicht angewandt werden. Beispiele dafür könnten Prothesen sein, die darauf ausgelegt wurden genau zu passen, oder Figuren, bei denen das Aussehen der wichtigste

Faktor ist.

2.5 5d-Druck

Der 5d-Druck ist eine Erweiterung des 3d-Drucks. Ein 3d-Drucker kann die Düse in der x und y Richtung und die Plattform in der z Richtung bewegen. Ein 5d-Drucker kann zusätzlich die Plattform in zwei Dimensionen kippen. Durch das 5d drucken können Schwachpunkte im Material verhindert werden und die generelle Materialstärke um das drei bis fünffache verbessert werden. Ein weiterer Vorteil ist die Nutzung von weniger Material [RD18]. Obwohl der 5d Druck eine vergleichsweise neue Technologie ist, gibt es bereits Anwendungsbereiche in der Medizin, für unter anderem die Produktion von Implantaten [Kum+26].

In der Abbildung 2.5 ist eine Düse und eine Plattform zu sehen. Die Plattform ist sichtlich gekippt. Die Fähigkeit die Plattform zu kippen ermöglicht es Bauteile zu drucken, ohne Stützstrukturen zu benötigen, da wenn eine Schicht am Überhängen ist, das Bauteil gedreht werden kann, sodass die nächste Schicht nicht mehr über die aktuelle Schicht überhängt. Durch das Kippen der Plattform ist es ebenfalls möglich Schichten, die nicht parallel zu der Plattform sind zu drucken. Die bessere Anpassung an die Geometrie der Bauteile verbessert die Festigkeit und Qualität des Bauteils. Keine Stützstrukturen zu benötigen reduziert den Materialaufwand und die Druckzeit. Die Bestimmung einer Ausrichtung des Bauteils ist eine wichtige und komplexe Bewegung im 3d Druck. Ein Bauteil mehrmals während des Druckes auszurichten zu können löst oder vereinfacht mehrere der Optimierungsprobleme des 3d Drucks, fügt aber auch zusätzliche Komplexität zu der Vorverarbeitung hinzu.

Diese Fähigkeit Schichten zu erstellen, die nicht parallel zu der Plattform sind, hat den Nachteil, dass die Berechnung der Schichten komplexer wird. Die meisten Programme zu Planung eines Druckes unterstützen keinen 5d-Druck, es gibt aber bereits experimentelle Ansätze für die lokale Ausrichtung einer Oberfläche [Hal+24]. Die Bewegungen auf allen Achsen zu koordinieren benötigt hohe Präzision und Planung. Im Gegensatz zu einem 3d Drucker kann das Drucken nicht Schicht um Schicht gemacht werden. Die Drehung der Baufläche oder der Düse sorgt dafür, dass die Schichten nicht immer direkt aufeinander folgen können oder immer parallel zueinander sind. Andere Einschränkungen bei der Erstellung von dem Düsenpfad für den 5d-Druck sind unter anderem die Kollisionsvermeidung, die Berechnung der optimalen Kippwinkel für jede Schicht und die Ablage von Material auf gekrümmten Flächen. Diese

⁵https://www.3dnatives.com/de/wp-content/uploads/sites/3/150217_Zurich1.jpg

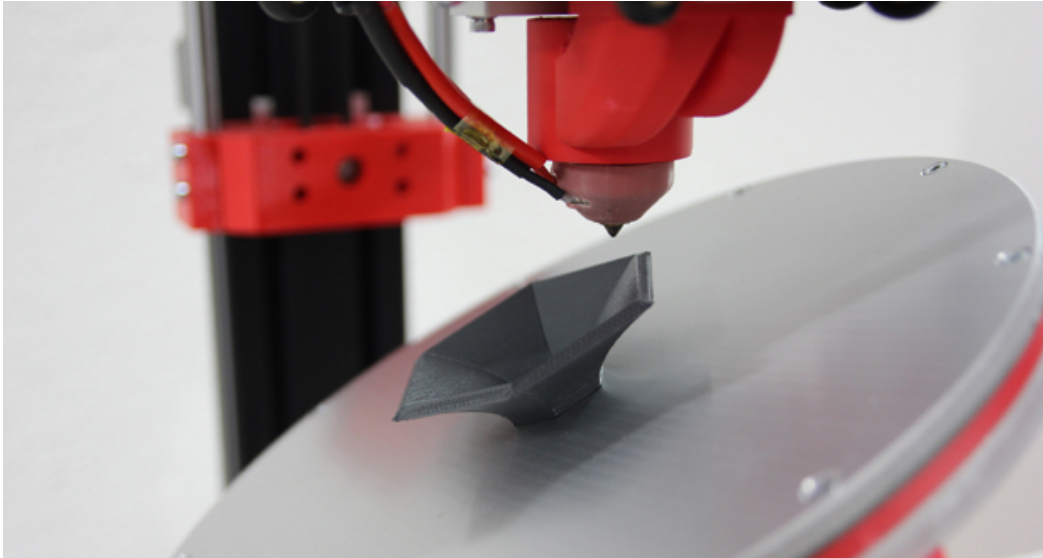


Abbildung 2.5: Darstellung der beweglichen Plattform eines 5d-Druckers⁵

Einschränkungen machen die Planung des Pfades, den die Düse bei dem Druckvorgang abfährt, komplexer als die Bewegung des Pfades für den 3d Druck [Kum+26]. Auf gekrümmten Flächen ist es schwer Material zu platzieren. Mögliche Lösungsansätze sind bessere Kontrolle der Düse verbesserte Haftung zwischen Schichten oder optimale und stabile Temperatur des Druckmaterials beim Austritt aus der Düse [Kum+26].

Die Kollisionsvermeidung ist ein Problem, das durch den 5d-Druck entsteht. Wenn das Bauteil sich während des Druckes drehen kann, besteht die Möglichkeit, dass die Düse mit dem Bauteil kollidiert. Bei dem 3d-Druck ist das nicht möglich, da die Düse immer über dem Druckbereich ist. Es macht die Planung des Düsenpfades schwerer, da bei der Reihenfolge der zu druckenden Flächen berücksichtigt werden muss, dass eine Fläche unter Umständen nicht erreichbar ist, wenn sie zu spät gedruckt wird. Optimale Kippwinkel für jede Schicht zu bestimmen, benötigt die Berechnung des optimalen Winkels für jede dieser Schichten. Dieser Prozess wird erschwert, da berücksichtigt werden muss, dass eine Fläche unter Umständen nicht von einem bestimmten Winkel aus erreichbar sein könnte, weil dort bereits etwas anderes gedruckt wurde, was mit der Düse kollidieren würde. Es muss also auch bei der Ausrichtung des Bauteils Kollisionsvermeidung berücksichtigt werden.

Der 5d Druck bietet die Möglichkeit komplexere Komponenten mit weniger Material zu drucken. Außerdem sind 5d gedruckte Teile strukturell stabiler. Allerdings ist es eine neue Technologie und es gibt noch mehrere technische Probleme die gelöst werden müssen. Ein großes Problem ist es, alle notwendigen Ausrichtungen möglichst

schnell zu berechnen. Es gibt bis jetzt noch keine weitgehend anerkannten oder leicht verfügbaren Lösungswege für dieses Problem.

3 Flächensuche in Bauteilen

3.1 Kriterien für geeignete Flächen

Bevor ein Bauteil ausgerichtet werden kann, müssen zuerst zusammenhängende Flächen, die möglichst geringe Krümmung aufweisen, im Bauteil identifiziert werden. Solche Flächen zu finden ist das Thema dieses Kapitels. Ein Bauteil ist ein Objekt, das gedruckt werden könnte, wie zum Beispiel ein Würfel. Die meisten Bauteile in dieser Arbeit sind künstlich generiert und nur für die Weiterverarbeitung oder zum Testen da. Sie sind keine Bauteile, die man in der Praxis drucken würde. Die Verarbeitung der gefundenen Flächen wird in Kapitel 4 behandelt. Die Suche der Fläche ist ausschließlich für die Weiterverarbeitung in Kapitel 4 da. Sie sind ein Zwischenergebnis.

Eine kohärente Fläche ist eine stetige Fläche. Sie hat weder Sprünge noch Lücken oder Risse. Diese Eigenschaft ist hilfreich, da eine Fläche die nicht kohärent ist, definitionsgemäß an mindestens einer Stelle einen Sprung oder Lücke haben muss. Es ist einfacher, wenn diese Fläche als zwei oder mehr Flächen weiterverarbeitet wird, anstelle von einer Fläche, die an mindestens einer Stelle getrennt ist. Eine zusammenhängende Fläche ist in dieser Arbeit eine Fläche, die entweder keine oder eine geringe Krümmung aufweist. Eine Krümmung kann nur bis zu einem bestimmten Punkt akzeptiert und ein Knick innerhalb einer Fläche sollte immer vermieden werden. Wie der Punkt bis zu dem eine Krümmung akzeptabel ist, festgelegt wird, wird in dem folgenden Abschnitt 3.2 erklärt. Eine Fläche mit einer Krümmung kann eine gute Ausrichtung unmöglich machen. Das Ziel der Ausrichtungen ist es eine Fläche senkrecht in Relation zu der Druckerplatte zu Orientieren. Bei einer gekrümmten Fläche ist dies unmöglich. Wenn man zwei Geraden durch zwei zufällige Punkte auf einer gekrümmten Fläche zieht und die beiden Geraden die Steigung der Fläche an den Punkten, durch die sie gehen, haben, dann werden sie nicht parallel sein. Geraden, welche die Steigung an einem Punkt darstellen, werden Tangenten genannt. Die Abbildung 3.1 zeigt eine gekrümmte Fläche in Blau und auf der Fläche zwei Tangenten, t_1 und t_2 , die in Magenta markiert sind. Es lässt sich erkennen, dass

es unmöglich ist zwei Tangenten zu wählen, die parallel sind und nicht vom selben Punkt ausgehen. Wenn die y-Achse die Grundfläche eines Druckers ist, dann lässt es sich ebenfalls erkennen, dass die Fläche unmöglich Senkrecht zu der Druckerfläche sein kann. Sobald in einer Fläche zwei Tangenten, die nicht parallel zueinander sind, existieren, ist eine perfekt senkrechte Ausrichtung unmöglich. Eine Krümmung sollte dementsprechend so gering wie möglich sein.

Dieses Kapitel erklärt, wie in einem Bauteil kohärente und zusammenhängende Flächen gefunden werden. Dafür wird ein Bauteil durch zufällig auf der Oberfläche verteilte Punkte $p \in \mathbb{R}^3$ dargestellt. Eine Menge $P = \mathbb{R}^{3 \times N}$ dieser Punkte wird Punktwolke genannt. Es werden in dieser Arbeit mehrere tausend Punkte benutzt, um ein Bauteil mit einer Punktwolke darzustellen. Danach wird die Punktwolke in mehrere kleinere Gruppen aufgeteilt.

Eine Aufteilung von Datenpunkten in Gruppen wird Clustering genannt, eine Gruppe ist ein Cluster. Die Aufteilung in Gruppen basiert auf den Werten der Datenpunkte, Daten mit ähnlichen Werten kommen generell in dieselbe Gruppe. Die Gruppen werden auch Cluster genannt. Was Ähnlichkeit bedeutet ist unterschiedlich, aber ein häufiges Ähnlichkeitsmaß und das Ähnlichkeitsmaß, welches in dieser Arbeit benutzt wird, ist die euklidische Distanz. Die euklidische Distanz ist die Distanz zweier Punkt in einer Ebene oder in einem Raum. Die Distanz, die sie beschreibt, ist die Länge der minimalen Strecke zwischen zwei Punkten. Wenn $p = (p_1, p_2, \dots, p_i)$ und $q = (q_1, q_2, \dots, q_i)$ Koordinaten in einem i-Dimensionalen Raum sind, dann ist die euklidische Distanz gleich $\sqrt{\sum_{i=1}^n (q_i - p_i)^2}$.

In der Abbildung 3.2 sind zwei zweidimensionale Punktwolken zu sehen. Die linke ist nicht geclustert, die rechte ist geclustert, aber beide stellen dieselbe Punktwolke dar. Die rechte Punktwolke ist mit drei Farben markiert, Punkte derselben Farbe sind im selben Cluster. In der rechten Punktwolke ist das Ergebnis von einem Clustering-Verfahren dargestellt. Clustering ist ein Verfahren, mit dem ein Satz an Daten in Gruppen eingeordnet wird. Die Elemente einer Gruppe sind sich innerhalb ähnlicher, als Elemente aus anderen Gruppen. Es lässt sich auf der rechten Seite der Abbildung 3.2 erkennen, dass alle Punkte einer Farbe nahe beieinander sind. Außerdem kann man eine klare Trennung zwischen den Farben erkennen. Es gibt keine Ausnahmen, wo ein Punkt der einen Farbe in dem Bereich einer anderen Farbe ist. Es lässt keine Ausnahmen zu. Clustering wird benutzt, um Konzentrationen von ähnlichen Punkten zu finden.

¹<https://i.stack.imgur.com/cIDB3.png>

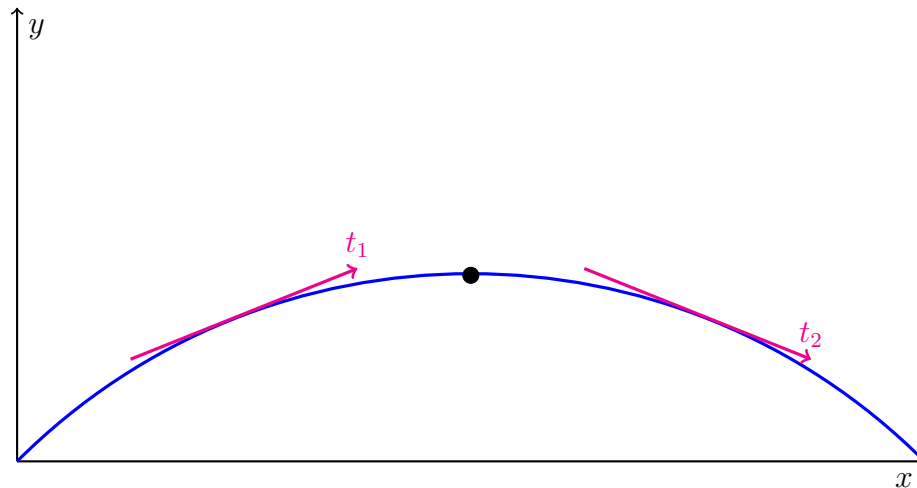
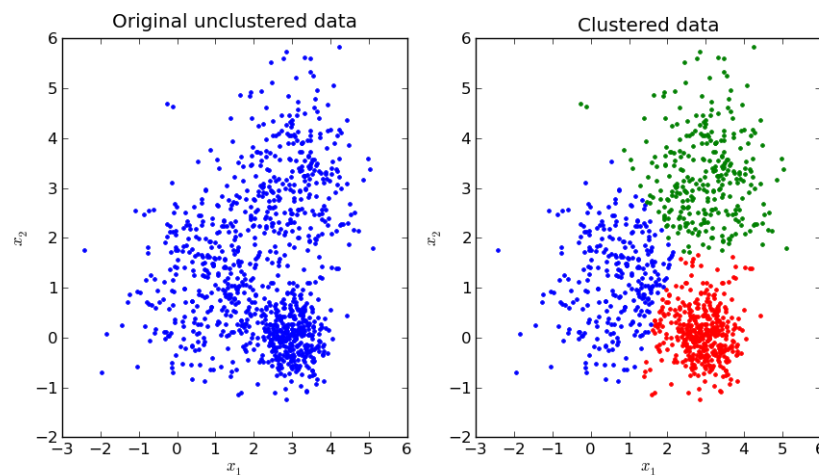


Abbildung 3.1: Darstellung von Tangenten in einer gekrümmten Fläche

Abbildung 3.2: Clustering von verstreuten zweidimensionalen Punkten¹

3.2 Clustering nach Normalvektoren in Bauteilen

3.2.1 Normalvektoren

Um zu messen, ob zwei Punkte Teil derselben Fläche sein sollen, muss die Ausrichtung eines Punktes gemessen werden. Punkte mit einer ähnlichen Ausrichtung gehören zu derselben Fläche. Mit der Ausrichtung eines Punktes ist ein Vektor, der senkrecht zu der Oberfläche an der Stelle des dazugehörigen Punktes steht, gemeint. Solche Vektoren werden Normalvektoren genannt. Es sind nur die Normalvektoren relevant, die einen dazugehörigen Punkt in der Punktwolke des Bauteils haben. In diesem Abschnitt werden Normalvektoren und ihre Relevanz für die Flächensuche erklärt. Auf ihnen wird die erste Aufteilung eines Bauteils in Abschnitt 3.2.2 durch-

geführt.

In der Abbildung 3.3 ist die Bauteiloberfläche blau markiert. Der Punkt p_i ist ein beliebiger Punkt auf der Bauteiloberfläche. Der Normalvektor n_i geht von dem Punkt p_i aus senkrecht nach oben. Er steht im 90 Grad Winkel zu der Bauteiloberfläche. Die Normalvektoren in der Abbildung 3.3 sind zweidimensional, im Gegensatz dazu sind die Normalvektoren auf den Punkten im Bauteil dreidimensional. Die Normalvektoren werden für jeden Punkt anhand der Oberfläche des Bauteils bestimmt.

Eine Aufteilung der Normalvektoren in Cluster ist notwendig, um Flächen zu erkennen. Dafür werden die Normalvektoren als dreidimensionale Punkte dargestellt, die euklidische Distanz zwischen den Normalvektoren gibt ihre Ähnlichkeit an. Punkte mit ähnlichen Normalvektoren haben eine ähnliche Ausrichtung und sollten Teil derselben Fläche sein. Die Darstellung als dreidimensionale Punkte funktioniert nur, wenn alle Normalvektoren normiert sind. Ein normierter Vektor hat eine Länge von 1.

3.2.2 Hierarchisches Clustering

Die in Abschnitt 3.2.1 beschriebenen Normalvektoren werden durch ein Clustering-Verfahren in diesem Abschnitt in Gruppen unterteilt. Die dadurch entstehenden Cluster enthalten jeweils Punkte mit ähnlichen Normalvektoren. Ein Cluster, das Punkte mit ähnlichen Normalvektoren enthält, bildet eine Fläche mit geringer Krümmung. Der Sinn des Clusterings eines Bauteils nach seinen Normalvektoren ist es demnach, das Bauteil in mehrere Flächen mit jeweils möglichst geringer Krümmung pro Fläche zu unterteilen.

Eine Aufteilung von Normalvektoren in Cluster basierend auf ihrer Ähnlichkeit lässt mehrere Möglichkeiten der Umsetzung offen. Durch Testen hat sich hierarchische Clustering als die beste Option herausgestellt. Methode, die einen Zielwert für die Anzahl an Cluster vor der Ausführung benötigen, haben sich als unzuverlässig erwiesen. Die feste Anzahl an Cluster die ausgegeben werden muss, macht solche Methoden nicht sehr anpassungsfähig. Bei Bauteilen mit einfacher Form, wie zum Beispiel ein Würfel, sollte wenige Flächen gefunden werden. In vielen Fällen wurden offensichtlich zusammenhängende Flächen in zwei geteilt, weil das Clustering verfahren gezwungen war die festgelegte Anzahl an Clustern zu finden. Bei komplexen Bauteilen würden dagegen viele Flächen zu erwarten sein. Es wurden Flächen die getrennt sein sollten nicht getrennt, da die Clustering Methode die festgelegte Anzahl an Clustern nicht überschreiten darf.

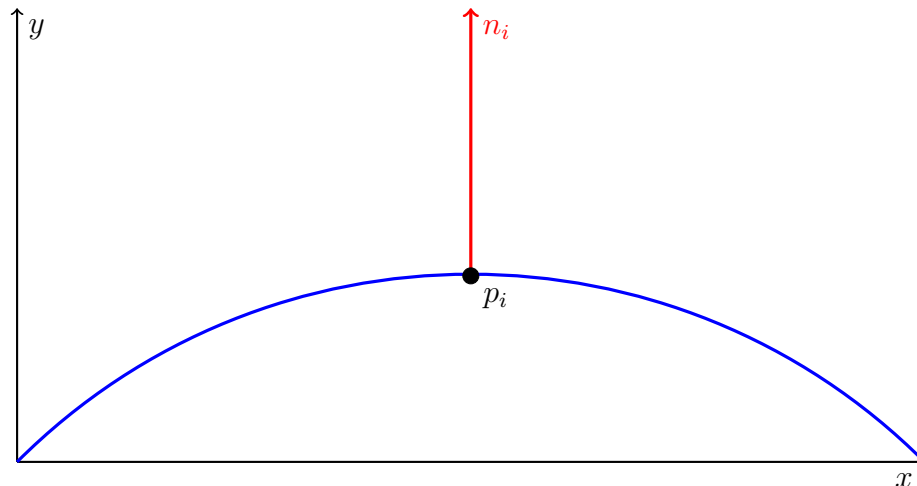


Abbildung 3.3: Darstellung von Normalvektoren an einem Punkt der Bauteiloberfläche

Dichte basierte Verfahren, wie DB-Scan, benötigen keine Angaben zu der Clusteranzahl vor der Ausführung. Sie haben sich aber ebenfalls als unzuverlässig erwiesen. Die Normalvektoren liegen dicht zusammen, sodass dieses Verfahren meistens entweder das gesamte Bauteil als ein Cluster oder nur Cluster, die zu klein sind, um nutzbar sein erkannt hat.

Hierarchische Clustering benötigt keine Anzahl an Clustern die gefunden werden sollen. Stattdessen wird eine Methode genutzt, in der alle Datenpunkte zum Start ihre eigene Gruppe haben, sodass es für jeden Datenpunkt eine Gruppe gibt. Die zwei Gruppen, die sich am ähnlichsten sind, werden dann zusammengefügt. Auf diesen Gruppen wird der Vorgang wiederholt, bis das Zusammenfügen von Gruppen immer einen vor der Ausführung festgelegten Differenzwert überschreitet. Der Grenzwert der nicht überschritten werden darf, ist die Unterschiedlichkeit der Normalvektoren in den Clustern. So passt sich die Anzahl der Cluster dem Bauteil an und es wird gleichzeitig sichergestellt, dass Normalvektoren in einem Cluster einen festlegbaren Wert an Ähnlichkeit nicht unterschreiten.

Die Abbildung 3.4 stellt das Vorgehen von Hierarchie Clustering dar. Dabei ist die y-Achse der Distanzwert und auf der x-Achse sind die Cluster markiert. Das Clustering fängt unten, bei $y = 0$ an. Dort ist jeder Punkt ein Cluster. Danach wird die Distanz zwischen allen Clustern gemessen und die zwei ähnlichsten Cluster werden zu einem Cluster zusammengefügt. Dieses Vorgehen wird so lange wiederholt, bis ein bestimmter Distanzwert zwischen allen Clustern überschritten wurde. In der Abbildung 3.4 kann man das Zusammenfügen von Clustern sehen, wenn zwei über eine senkrechte Linie verbunden werden. Aus der Linie, welche die beiden Cluster verbindet, kommt eine weitere Linie, die das neue zusammengefügte Cluster symbo-

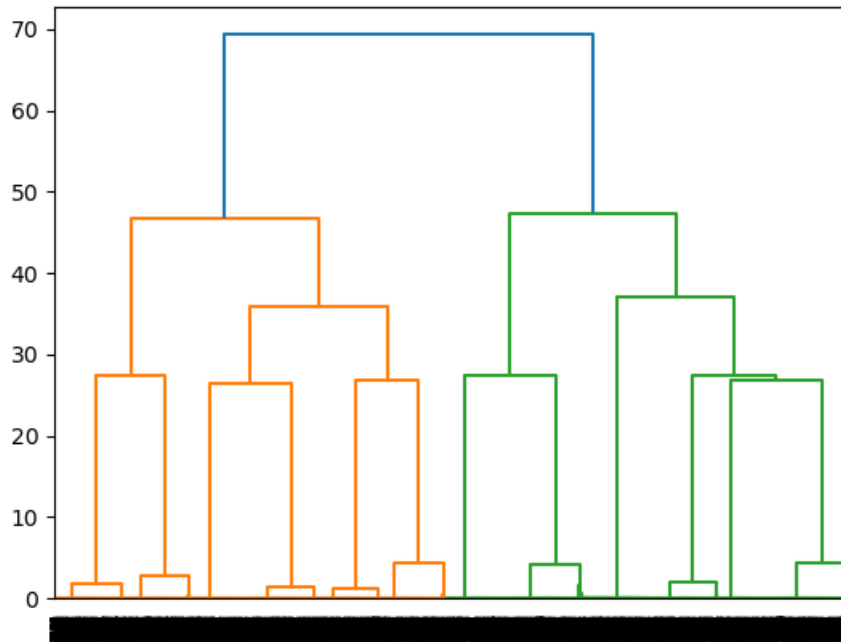


Abbildung 3.4: Beispiel von Hierarchie Clustering als Dendrogramm

liert. Das hat den Vorteil, dass die Anzahl der Cluster in einem Bauteil von dem Bauteil selber abhängt. Es ist nicht nötig eine feste Anzahl an Clustern festzulegen, sondern nur eine Ähnlichkeit, welche die Cluster haben sollen. In der Abbildung 3.4 stoppt der Algorithmus nicht, nachdem alle Cluster einen vorbestimmten Distanzwert überschreiten, sondern fügt so lange Cluster zusammen, bis nur noch eins übrig ist.

Das Zusammenfügen von Gruppen basiert auf Ähnlichkeit der Gruppen. In jedem Schritt werden die ähnlichsten Gruppen zusammengefasst. Die Ähnlichkeit zwischen Gruppen wird über die Ward Methode bestimmt [War63]. Diese Methode vereinigt die zwei Gruppen, mit dem geringsten Anstieg an Varianz. Die Varianz von einer Gruppe an Punkten ist die durchschnittliche Abweichung von dem Zentrum der Punkte. In diesem Fall wird die quadratische Varianz genommen, also die durchschnittliche Abweichung zum Quadrat. Für jede Gruppe G wird die Varianz mittels $\sum_{i \in G} (\|\bar{x} - x_i\|^2)$ bestimmt. $\bar{x}$ ist dabei der Durchschnitt aller Punkte in G . Der Veränderung der Varianz bei dem Zusammenfügen zweier Gruppen A, B kann durch $\sum_{i \in A \cup B} (\|\bar{x} - x_i\|^2) - \sum_{i \in G} (\|\bar{x} - x_i\|^2) - \sum_{i \in G} (\|\bar{x} - x_i\|^2)$ bestimmt werden. Von der Varianz der zusammengeführten Gruppen wird die Varianz der beiden einzelnen Gruppen abgezogen, das Ergebnis ist die Erhöhung der Varianz durch das Zusam-

menfügen von A und B. Die Gruppen zwischen denen diese Erhöhung minimal ist, werden zusammengefasst.

Auf diese Art und Weise werden die Cluster zwischen Normalvektoren gesucht. Alle Normalvektoren, die in einem Cluster sind, sind so ähnlich, dass das hierarchische Clustering sie nicht weiter zusammengefügt hat. Der Distanzwert auf der y-Achse in 3.4 dient dabei als Fehlermaß für die Flächen. Wenn das Fehlermaß null ist, werden nur Flächen gefunden, die absolut Flach sind. Aber bei komplexen Bauteilen ist es notwendig einen höheren Fehler zuzulassen, um gebogene Flächen zu finden. Bei komplexen Bauteilen würde die Suche nach ausschließlich gerade Flächen in sehr vielen, extrem kleinen Flächen resultieren, da viele Bauteile keine geraden Flächen haben werden. Eine Abweichung zuzulassen ist also sinnvoll.

Das Clustering nach Normalvektoren ist nur geeignet um Flächen zu finden, die, innerhalb des Distanzwertes als Fehlermaß, flach sind. Das Clustering gibt keine Garantie, dass die gefundenen Flächen kohärent sind. In Abbildung 3.5 ist sichtbar, dass die in Grün markierten Flächen, parallel zueinander, aber nicht kohärent miteinander, sind. Die Flächen die in Grün markiert sind, sind Teil desselben Clusters, aber sollten es nicht sein, da sie eindeutig voneinander getrennt sind. Die roten Punkte markieren das Bauteil, in dem die Flächensuche durchgeführt wurde.

Der nächste Abschnitt 3.3 beschäftigt sich damit, wie die in den hierarchischen Clustering gefundenen Flächen getrennt werden.

3.3 Separation von parallelen Flächen

Alle Flächen in einem Cluster des Normalvektorclusterings aus Abschnitt 3.2 sind in etwa parallel und möglichst gerade, aber unter Umständen räumlich voneinander getrennt. Das macht sie ungeeignet für die Bildung einer Ausrichtung in Kapitel 4. Sie werden in diesem Abschnitt mit einem weiteren Clustering-Verfahren getrennt, sodass nur zusammenhängende Flächen ohne Sprünge oder Lücken das Ergebnis sind.

In Abbildung 3.5 ist eine beispielhafte Ausführung des Clusterings auf den Normalvektoren, welches in Abschnitt 3.2.2 beschrieben wird, dargestellt. Die grünen Punkte sind die ein Cluster, dass von dem Normalvektorclustering gefunden wurde. Die roten Punkte sind die übrigen Punkte des Bauteils. Punkte, die erkennbare Flächen bilden, sind immer sehr dicht beieinander. Zwischen den erkennbaren Flächen sind große Lücken ohne Punkte. Die Dichte der Punkte kann somit genutzt werden, um die einzelnen Flächen voneinander zu trennen.

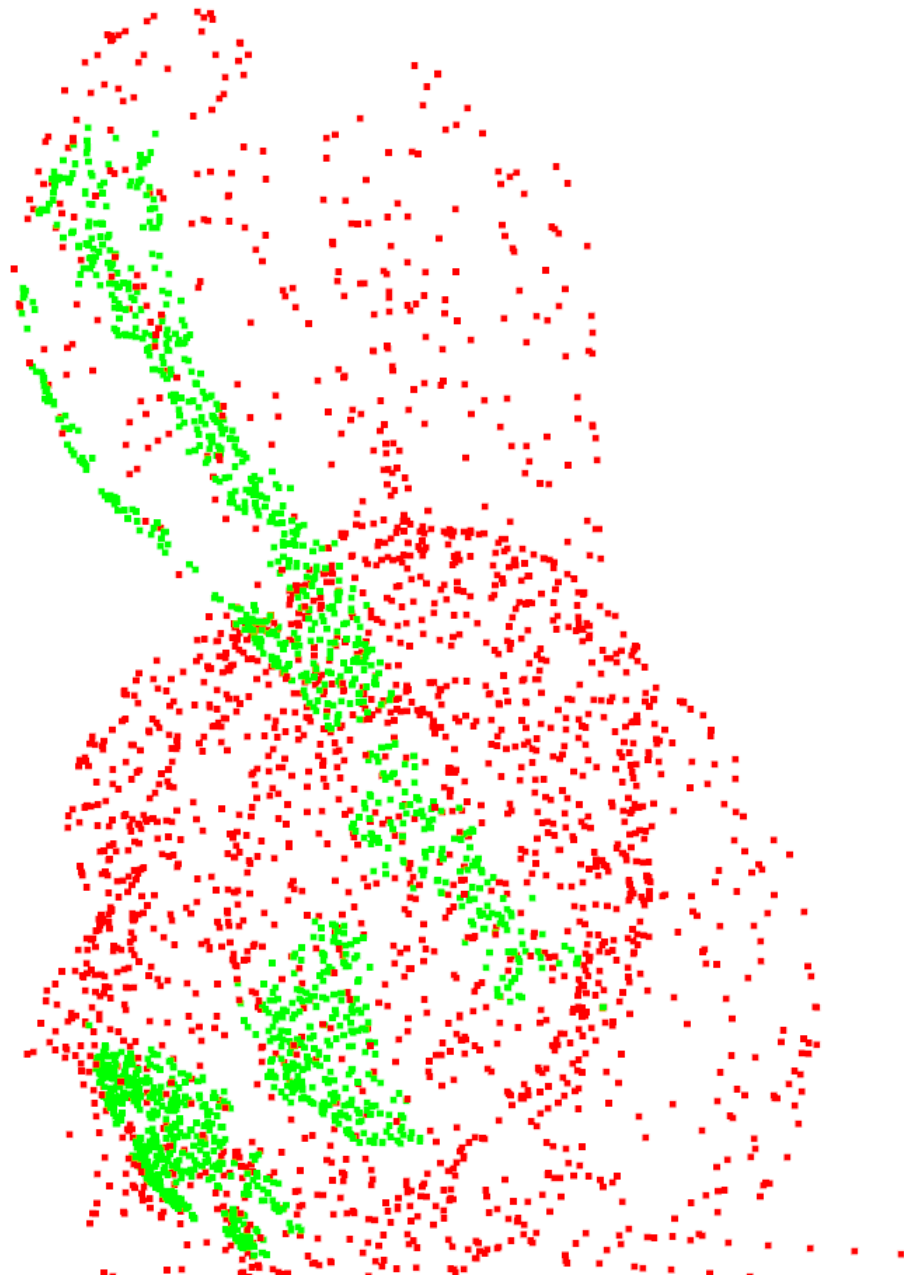


Abbildung 3.5: Hierarchisches Clustering eines Bauteils nach Normalvektor

DB-Scan ist eine auf Dichte basierende Clustering Methode. Diese Methode braucht wenig Informationen über die zu gruppierenden Daten, funktioniert auf Daten jeglicher Form und ist auch auf großen Datensätzen effektiv [Est+96].

Die räumliche Trennung zwischen den Flächen in einem Cluster kann genutzt werden, um sie zu trennen. Dafür wird der DB-Scan Algorithmus verwendet. DB-Scan benötigt einen Radius und eine Mindestanzahl an Nachbarn. Der Radius wird *esp* und die Mindestanzahl an Nachbarn wird *minPts* in Abbildung 3.6 genannt. Die

Eingabe muss relativ zu der Dichte der Flächen bestimmt werden.

Die Abbildung 3.6 zeigt ein Beispiel für die Funktion von dem DB-Scan Algorithmus, der im Folgenden erklärt wird. Der Algorithmus nimmt einen Punkt, der in keinem Cluster ist und überprüft, wie viel Punkte in einem Radius esp um ihn herum sind. Wenn innerhalb des Radius mindestens $minPts$ viele Punkte sind, dann ist der Punkt ein Kernpunkt (engl. Core Point) eines neuen Clusters. Alle Punkte, die in dem Radius des Punktes sind, werden Teil des Clusters und genauso überprüft, wie der Startknoten des Clusters. Dieses Vorgehen wird fortgeführt, bis keine Punkte mehr hinzugefügt werden können. Wenn ein Punkt Teil des Clusters ist, aber nicht mindestens $minPts$ viele Punkte in seiner Nachbarschaft hat, dann ist dieser Punkt ein Randpunkt (engl. Borderpoint), aber immer noch Teil des Clusters. Wenn weniger als $minPts$ innerhalb des Radius sind, dann gilt der Punkt als Rauschpunkt (engl. Noise). Das bedeutet aber nicht unbedingt, dass der Punkt Noise bleibt. Er kann auch zu einem Randpunkt eines Noch nicht existierenden Clusters werden. Es werden so lange neue Punkte außerhalb von Clustern gewählt, bis alle Punkte Teil eines Clusters oder Noise sind. Die Reihenfolge, in der die Punkte abgearbeitet werden, ist größtenteils irrelevant. Die Randpunkte der Cluster können durch die Reihenfolge der Ausführung beeinflusst werden, die Kernpunkte der Cluster bleiben dagegen unverändert.

Die Algorithmen 3.1 und 3.2 stellen die Arbeitsweise von DB-Scan mit Pseudocode dar.

Der DB-Scan Algorithmus 3.1 geht alle Punkte einer Punktwolke durch und sucht nach Punkten, die noch nicht zugewiesen wurden. Nachdem alle Punkte ein Label bekommen haben, ist der Algorithmus fertig. Für jeden Punkt, der aktuell unbestimmt ist, wird der Unteralgorithmus 3.2 aufgerufen, um zu bestimmen, ob der unbestimmte Punkt ein Cluster bilden kann. Der Unteralgorithmus ClusterErweitern überprüft, wie viele Nachbarn der unbestimmte Punkt hat. Wenn es mindestens $minPts$ viele sind, ist ein neues Cluster gefunden worden. Alle Nachbarn des Punktes sind ebenfalls Teil des Clusters. Auf ihnen wird der ClusterErweitern Algorithmus erneut ausgeführt. Wenn sie ebenfalls mindestens $minPts$ innerhalb des esp Radius haben, werden sie auch alle Teil des Clusters und der Vorgang wird erneut wiederholt. Wenn ein Punkt die ClusterID eines Clusters hat, aber nicht die Mindestanzahl an Nachbarn hat, dann ist es ein Randpunkt. Auf den Nachbar eines Randpunktes wird der ClusterErweitern Algorithmus nicht ausgeführt und sie werden auf nicht zum Teil des Clusters gemacht, außer sie waren es schon vorher. Randpunkte sind

²<https://machinelearninggeek.com/wp-content/uploads/2020/10/image-58.png>

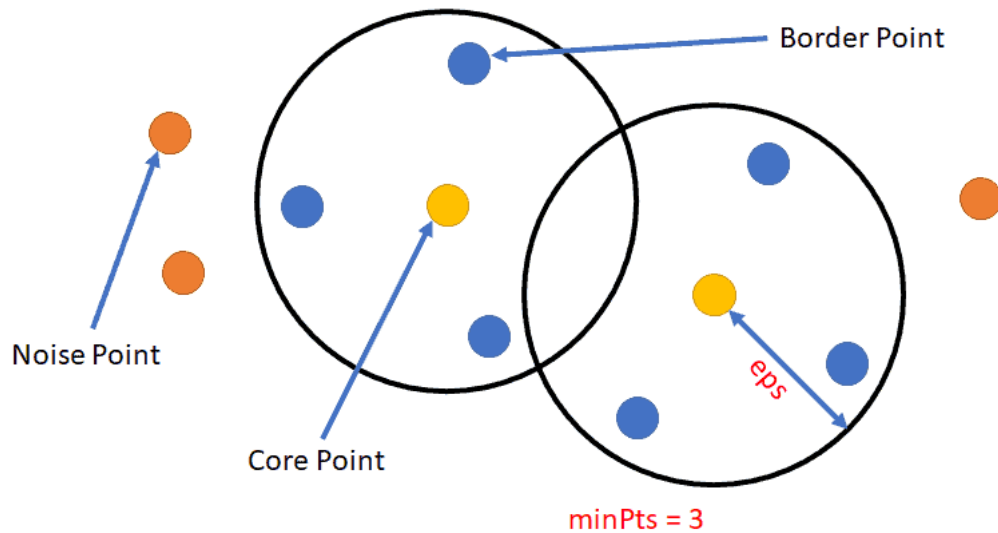


Abbildung 3.6: Beispielhafte Darstellung von DB-Scan Algorithmus²

dennoch Teil eines Clusters und behalten Ihre ClusterID. Auf diese Art und Weise wird ein Cluster erweitert, bis es keine Punkte mehr gibt die Hinzugefügt werden können.

Falls der ClusterErweitern Algorithmus nicht mindestens minPts Nachbar von dem Startpunkt findet, dann gibt ClusterErweitern False zurück und der Punkt wird als Rauschpunkt gekennzeichnet.

Alle Flächen innerhalb eines Cluster der Normalvektoren sind räumlich getrennt. Wenn der DB-Scan Algorithmus auf die vorher gefunden Cluster angewandt wird, findet er kohärente Flächen als Cluster: In diesen Flächen liegen die Punkte sehr nahe beieinander, wodurch die Punkte in der Fläche meistens die minPts Bedingung erfüllen. Andere Flächen werden wahrscheinlich nicht innerhalb von eps der Randpunkte einer Fläche liegen. Es gibt keine Garantie, dass alle Flächen getrennt werden, aber in den meisten Fällen wird es passieren. Ein kleinerer Radius verringert die Chance die Trennung von zwei Flächen zu übersehen, aber benötigt auch mehr Punkte, um verlässlich zu funktionieren. Mit zu wenigen Punkten könnte eine ununterbrochene Fläche fehlerhaft getrennt werden. Die Erhöhung der Punkte führt außerdem zu einer erhöhten Rechenzeit. DB-Scan ist auch auf großen Datensätzen effizient, mit einer Zeit Komplexität relativ zu der Eingabelänge von $O(n \log n)$ [GD13]. Davor muss das hierarchische Clustering aus Abschnitt 3.2.2 ausgeführt werden. Es hat eine Zeitkomplexität von $O(n^2)$ [DE84]. Mit einer quadratischen Zeitkomplexität ist das Clustern auf den Normalvektoren der Problempunkt, durch den die gesamte Flächensuche eine Zeitkomplexität von $O(n^2)$ hat.

Eingabe: Punktwolke, eps , $minPts$

```

 $i = 0$ 
for  $Punkt \in Punktwolke$  do
  if  $Punkt.ClusterID = Unbestimmt$  then
    if  $ClusterErweitern(Punktwolke, Punkt, ClusterID, eps, minPts)$  then
       $Punkt.ClusterID = neueClusterID$ 
    end if
  end if
end for

```

Algorithmus 3.1: DB-Scan Algorithmus

Eingabe: Punktwolke, $Punkt$, $ClusterID$, eps , $minPts$

```

 $nachbarn = regionsSuche(Punkt, eps)$ 
if  $nachbarn.size \geq minPts$  then
  for  $kandidat \in nachbarn$  do
    if  $kandidat.ClusterID = UnbestimmtORRauschpunkt$  then
       $RETURN ClusterErweitern(Punktwolke, kandidat, ClusterID, eps, minPts)$ 
    end if
  end for
else
  if  $Punkt.ClusterID \neq Unbestimmt$  then
     $kandidat.ClusterID = ClusterID // Randpunkt$ 
     $RETURN True$ 
  else
     $Punkt.ClusterID = Rauschpunkt$ 
     $RETURN False$ 
  end if
end if

```

Algorithmus 3.2: ClusterErweitern Algorithmus

Zuletzt werden von den finalen Clustern alle aussortiert, die eine bestimmte Punktzahl unterschreiten. Flächen mit zu wenigen Punkten würden schwer zu verarbeiten sein, daher ist es einfacher, sie nicht zu betrachten. Der Verlust an Punkten dadurch wird nur gering sein. Für eine echte Teilausrichtung dürften allerdings keine Punkte verloren gehen. Jeder Punkt muss gedruckt werden. Das Problem ließe sich im Realfall lösen, indem das Aussortieren weggelassen wird.

Das Ergebnis das durch das zweifache Clustern erreicht wurde, ist die Verarbeitung von Bauteilen zu zusammenhängenden und kohärenten Flächen, die in einem Bauteil gefunden werden können.

4 Punktwolkenverarbeitung zur Bauteilausrichtung

4.1 Ziel der Punktwolkenverarbeitung

In diesem Kapitel wird dargestellt, wie man eine Punktwolke, die eine kohärente und zusammenhängende Fläche in einem Bauteil darstellt, zu einer Rotationsmatrix verarbeiten kann. Für die Verarbeitung wird ein neuronales Netz eingesetzt, welches im Folgenden genauer erklärt wird. Dieses neuronale Netz wird eine Punktwolke beliebiger Größe zu der Z-Invarianz transformieren. Die Z-Invarianz wird in einem folgenden Abschnitt 4.3 zu einer Rotationsmatrix umgeformt. Mit dieser Rotationsmatrix lässt sich dann eine Drehung darstellen.

Um ein Bauteil korrekt auszurichten, muss eine Fläche des Bauteils auf eine Ausrichtung abgebildet werden. Die Flächen in den Bauteilen wurden in Kapitel 3 identifiziert und als Punktwolken gespeichert. Punktwolken wurden im Kapitel 3 beschrieben. Eine Punktwolke ist eine Menge an Punkten im 3D Raum, die zusammen eine Fläche oder ein Objekt darstellen. Die Ausrichtung die berechnet wird, wird Z-Invarianz genannt. Die Z-Invarianz ist ein R^3 Vektor, der die Rotation eines Bauteils beschreibt. Sie ist aber keine Rotationsmatrix und muss daher von einer Beschreibung der Drehung zu einer anwendbaren Rotationsmatrix transformiert werden. Die Z-Invarianz beschreibt wie ein Bauteil relativ zu der z-Achse rotiert werden soll. Wenn die Z-Invarianz senkrecht zu der z-Achse steht, dann ist die Fläche, die mit dieser Z-Invarianz rotiert wird, parallel zu der z-Achse. Die Fläche ist dann wie eine Wand ausgerichtet und hat dementsprechend wenig Überhang und lässt sich gut in Schichten unterteilen. Die Ausrichtung relativ zu den anderen Achsen wird nicht durch die Z-Invarianz beschrieben. Sie sind nicht für die optimale Ausrichtung relevant. Der Umweg über die Z-Invarianz ist notwendig. Der Versuch eine Rotationsmatrix direkt zu berechnen hat zu fehlerhaften Ergebnissen geführt. Das neuronale Netz hält sich nicht an die mathematischen Regeln einer Rotationsmatrix, wodurch Fehler entstehen. Nur die Z-Invarianz kommt daher als Repräsentation der

Rotation infrage. Die Z-Invarianz sollte im optimalen Fall im 90 Grad Winkel zu dem Normalvektor einer Fläche liegen, dadurch wird die Fläche möglichst parallel zu der Achse der Druckerdüse gedreht. Eine Fläche, die parallel zu der Düse des Druckers ausgerichtet ist, minimiert den Überhang, den die Fläche haben könnte. Der Normalvektor der Fläche ist der Durchschnitt über die Normalvektoren aller Punkte in der Fläche. Wie die Ausrichtung der Punkte bestimmt wird, wird in Abschnitt 3.2.1 erklärt. Wie aus der Z-Invarianz eine Rotationsmatrix erstellt wird, wird in Abschnitt 4.3 erklärt.

4.2 Rotationserstellung für Flächen

4.2.1 Maschinelles Lernen

Die Transformation der Punktwolken erfolgt über ein neuronales Netz. Ein neuronales Netz kann Daten verarbeiten, indem es sie durch viele Schichten an künstlichen Neuronen durchleitet. Dieser Abschnitt behandelt wie neuronale Netze Daten verarbeiten und über Zeit ihren Fehler reduzieren.

Ein künstliches Neuron ist eine Funktion, die als Eingabe andere künstliche Neuronen oder echte Daten nimmt. Die Eingabe des Neurons besteht aus den einzelnen Eingaben (x_i) multipliziert mit den Gewichten für die einzelnen Eingaben (w_i).

$$\sum_{i=1}^n x_i * w_i = o_j \quad (4.1)$$

Diese Funktion bildet die gewichteten Eingaben dann auf eine Ausgabe o_j mittels Aktivierungsfunktion ab, die entweder an ein weiteres Neuron übergeben wird oder die Ausgabe des Netzes darstellt. Die Abbildung in der Aktivierungsfunktion kann jede Funktion sein, die einen Wert x auf einen anderen Wert y abbildet, allerdings sind nicht lineare Funktionen wie zum Beispiel: $y = \max(0, x)$ (ReLU) generell besser geeignet, da man nur mit dieser Art von Aktivierungsfunktionen nicht lineare Zusammenhänge darstellen kann.

Ein typisches Neuron ist in Abbildung 4.1 dargestellt. Durch das Anpassen von Gewichten, w_i in der Abbildung 4.1, kann ein Neuron den Einfluss von Eingabedaten (x_i) verändern. Die optimale Belegung von Gewichten wird von dem Netz über mehrere Versuche hinweg erarbeitet. Dafür muss definiert werden, was das neuronale Netz als korrektes Ergebnis ansieht. Für die Anpassung der Gewichte in jedem

¹https://upload.wikimedia.org/wikipedia/commons/7/7f/ArtificialNeuronModel_deutsch.png

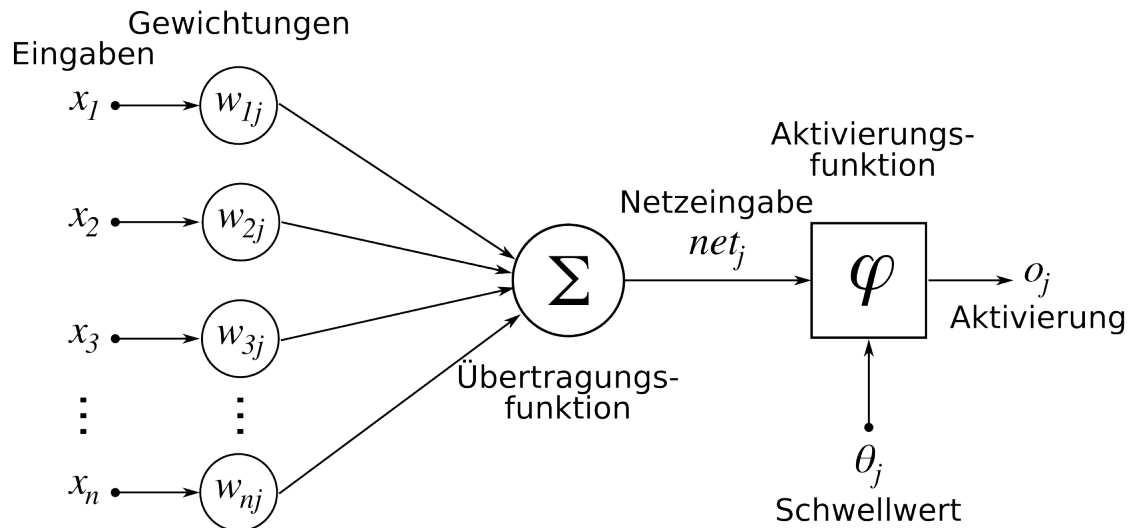


Abbildung 4.1: Funktionsweise eines künstlichen Neurons¹

Neuron wird ein Datensatz genutzt, den das neurale Netz mehrmals durchläuft. Diese Daten werden Trainingsdaten genannt. Bei jedem Durchlauf der Trainingsdaten überprüft das neurale Netz, wie gut seine Ausgaben im Vergleich zu dem vordefinierten korrekten Ergebnis ist. Die Trainingsdaten brauchen für jeden Datenpunkt ein richtiges Ergebnis, damit das neurale Netz sein Ergebnis mit dem vorher berechneten oder festgelegtem Optimum vergleichen kann. In dem Fall ist das richtige Ergebnis der Normalvektor zu der jeweiligen Punktwolke und das Netz nimmt die Standardabweichung von der Orthogonalität zu dem Normalvektor der Fläche als Fehler. Umso mehr die Z-Invarianz in einem 90 Grad Winkel zu dem Normalvektor der Fläche steht umso geringer ist der Fehler. Das Netz sieht jede Bewegung des Bauteils ebenfalls als Fehler. Umso größer die Bewegung desto größer der Fehler. Die beiden Fehler werden zusammenaddiert und ergeben dann den Fehler, den das Netz zu verringern versucht, indem es Gewichte anpasst. Der Fehler, den die Bewegung macht, wird aber vor der Addition künstlich verringert, um die Ausrichtung der Fläche als wichtigsten Aspekt des Netzes zu behalten.

Die Gewichte in den Neuronen werden vor der Anpassung zufällig festgelegt. In jeder Iteration werden die Gewichte an den Neuronen angepasst, falls dies zu einem besseren Ergebnis führt. Dafür wird über Gradienten berechnet, wie sehr jedes Gewicht zu dem Fehler beigetragen hat. Über viele Iterationen versucht das Netz dadurch den Fehler (engl. Loss) zu minimieren.

In vereinfachter Darstellung der Schichten eines neuronalen Netzes ist in Abbildung 4.2. Eine Schicht an künstlichen Neuronen ist so definiert, dass man die Neuronen

²https://miro.medium.com/1*LaEgAU-vdsR_pC1McgbikQ.jpeg

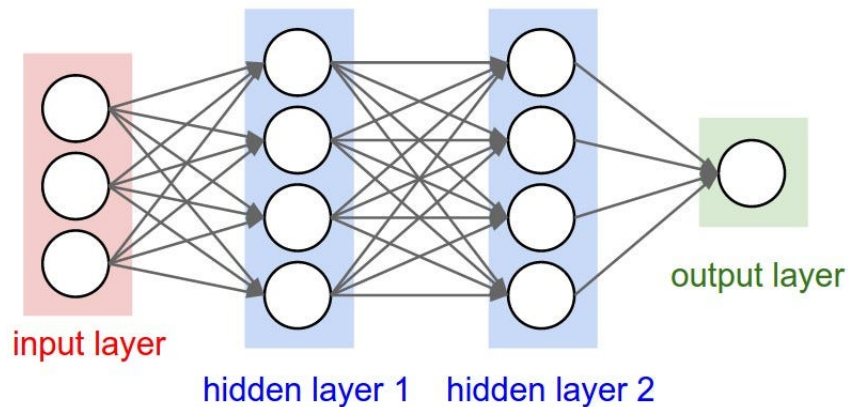


Abbildung 4.2: Schematische Darstellung von Regressionsnetz²

der Schicht immer mit der gleichen Anzahl an Kantenübergängen erreicht. Jedes Neuron der ersten verborgenen Schicht kann zum Beispiel mit nur genau einem Kantenübergang erreicht werden. Für eine vollständig verbundene Schicht muss jedes künstliche Neuron mit jedem künstlichen Neuron der vorherigen und darauffolgenden Schicht verbunden sein. Dabei gibt es drei Arten von Schichten, wie in der Abbildung 4.2.3 zu sehen ist. Zuerst gibt es die Eingabeschicht. Diese Schicht repräsentiert die Eingaben und ist in Abbildung 4.2 rot markiert. Jeder Knoten der Eingabeschicht ist ein Datenpunkt, oder in diesem Fall ein Merkmal. Danach gibt es die versteckten Schichten, in Blau markiert. Ihre Aufgabe ist es die Eingabe zu transformieren. Sie erlauben es dem neuronalen Netz eine Eingabe über mehrere Schichten zu verarbeiten. Durch kann das neuronale Netz nicht lineare Zusammenhänge darstellen. Zuletzt gibt es die Ausgabeschicht. Von dieser Schicht wird die Z-Invarianz abgelesen.

Nicht alle neuronalen Netze funktionieren über mehrere voll verbundene Schichten. Die Idee eine Eingabe schichtweise durch das Anpassen von Gewichten zu verarbeiten kann aber in fast allen gefunden werden. Das für diese Arbeit genutzte neuronale Netz reduziert erst die Eingabe auf Merkmale und nutzt dann mehrere voll verbundene Schichten um aus den Merkmalen eine Ausgabe zu generieren. Sie sind Darstellungen der Eingabedaten, die das neuronale Netz verstehen kann. Merkmale sind, wenn sie von einem neuronalen Netz gefunden wurden, für gewöhnlich extrem abstrakt und ohne viel Arbeit nicht für Menschen verständlich.

4.2.2 Merkmalsextraktion aus Punktwolke

Wenn eine Eingabe ohne Umwege auf die Ausgabe abgebildet werden soll, muss ein tiefes Netz eingesetzt werden. Um ein Bauteil automatisch auszurichten, ist es notwendig, dass das neuronale Netz die Eingabedaten selber verarbeitet. Der Unter-

schied zwischen einem tiefen und nicht tiefen neuronalen Netz ist es, dass ein tiefes neuronales Netz nicht nur die Erstellung einer möglichst korrekten Ausgabe erlernt, sondern auch die Extraktion von Merkmalen aus den Eingabedaten. Die Merkmalsextraktion nutzt als Eingabe eins der in Kapitel 3 gefundenen Cluster, die immer Flächen eines Bauteils sind. Zu den Punktwolken der gefundenen Flächen werden Merkmale erstellt. Das Finden von Merkmalen einer Fläche ist das Thema dieses Abschnittes. Eine Menge an Merkmalen, die zu einer Fläche gehören, stellen diese Eingabe in abstrakter und reduzierter Form dar. In der Merkmalsextraktion wird eine Punktwolke auf eine Anzahl an Merkmalen reduziert. Die Merkmale, auf welche die Punktwolke reduziert wird, werden in einem flachen Vektor $(v_0, v_1, v_2, \dots, v_n)$ mit $v_i \in \mathbb{R}$ der Größe n repräsentiert. Dieser Vektor wird Merkmalsvektor genannt und ist eine abstrakte Darstellung der Punktwolken. Der Merkmalsvektor ist als Zwischenergebnis wichtig, um die Eingabedaten in für ein Regressionsnetz besser verarbeitbare Form zu bringen. Für Menschen ist der Inhalt eines Merkmalsvektors meistens unverständlich. Das Regressionsnetz ist der zweite Teil der Struktur des Neurales Netzes und wird in dem folgenden Abschnitt 4.2.3 beschrieben.

Die Struktur der Merkmalsextraktion basiert der PointNet++ Architektur [Qi+17b]. Es wurde bereits die PointNet Architektur genutzt, um eine Bauteilausrichtung zu lernen [Zel+25]. Die Architektur wurde gewählt, weil sie vergleichsweise einfach und auf Punktwolken arbeiten kann. Es wurde genutzt, um Objekte auszurichten, sodass sie aufrecht stehen. Für das Problem der Flächenausrichtung hat es sich als unbrauchbar erwiesen. PointNet unterstützt keine Eingaben unterschiedlicher Größe. Die Punktwolken der Flächen sind Teilmengen der Punktwolke eines Bauteils. Es werden daher unterschiedlich viele Punkte für die Darstellung von Flächen genutzt. Die Flächen müssten alle auf die gleiche Größe gebracht werden, um durch PointNet verarbeitbar zu sein. Es ist möglich, alle Flächen auf eine Größe zu bringen, indem man mehr Punkte einer Fläche mehrmals zu der Punktwolke der Fläche hinzugefügt werden. Dieser Ansatz könnte aber unerwartete Probleme verursachen. Die PointNet++ Architektur kann Punktwolken unterschiedlicher Größe besser verarbeiten. In Tests zu dem Vergleich der Leistung der beiden Architekturen hat PointNet++ die generell besseren Ergebnisse erzielt. Für die Tests wurden Daten genutzt, die eine verlässliche Größe haben. So wurden Probleme bei der Eingabegröße vermieden.

PointNet++ benötigt keine Veränderungen zu den Flächen und erzielt bessere Ergebnisse. Es ist allerdings komplexer als PointNet und dementsprechend auch um ein Vielfaches rechen- und speicherintensiver.

Die generelle Struktur der Merkmalsextraktion ist eine wiederholte Folge an Redu-

zierungen auf den konkreten Daten der Punktwolke, gefolgt von einem kleinem voll verbundenen neuronalen Netz, was mit den konkreten Daten dieser Schicht Merkmale über mehrere Schichten hinweg konstruiert. In dem ersten Schritt in jeder Schicht werden Punkte aus der Punktwolke ausgewählt. Dafür wird ein Algorithmus eingesetzt, der eine Teilmenge der Ausgangspunkte auswählt, um eine Möglichst gute Abdeckung über die Ausgangspunkte zu erreichen [Eld+97]. Der Algorithmus wird Farthest Point Sampling genannt.

Der Algorithmus 4.1, für Farthest Point Sampling, ist als Pseudocode dargestellt. Zuerst wird ein zufälliger Punkt in die Menge der Ausgewählten Punkte hinzugefügt. Danach wird in jedem Schritt ein Punkt der Auswahl hinzugefügt. Es wird jeder Punkt der Punktwolke, der nicht schon Teil der Auswahl ist, mit jedem Punkt der Auswahl verglichen. Für jeden Punkt der Punktwolke wird die minimale Distanz zu einem Punkt der Auswahl berechnet. Der Punkt mit der aktuell größten minimalen Distanz zu einem den Punkten der Auswahl wird als Kandidat gespeichert. Sobald alle Punkte mit verglichen wurden und der finale Kandidat feststeht, wird der Kandidat zu den ausgewählten Punkten hinzugefügt. Der finale Kandidat ist der Punkt der Punktwolke, der den größten Abstand zu dem nächsten Punkt der Auswahl hat. Dieser Vorgang wird so lange wiederholt, bis die gewünschte Anzahl an Punkten n in der Auswahl enthalten ist. Es wird nicht nach der optimalen Abdeckung gesucht, welche die Distanz von jedem nicht ausgewählten Punkt zu dem nächsten ausgewählten Punkt minimiert, sondern ist nur eine Abschätzung, dieser optimalen Lösung.

Die gefundenen Punkte bilden die ausgewählten Punkte für die aktuelle Schicht. Danach werden für jeden ausgewählten Punkt jeder Punkt innerhalb eines Radius um sich selber verbunden. Diese Punkte sind die Nachbarn des ausgewählten Punktes. In der Abbildung 4.3 wird die Auswahl von Punkten und die Reduktion der ausgewählten Punkte in dem SSampling and GroupingSSchritt gezeigt. Es ist zu sehen, dass um bestimmte Punkte nach diesem Schritt ein Radius gezeichnet ist, der die Nachbarn der ausgewählten Punkte darstellt. Wenn die Auswahl der Punkte A ist und die gesamte Punktwolke P ist, dann gilt:

$$A \subseteq P, G = (V, E), \quad (4.2)$$

$$\forall \vec{a} \in A, \forall \vec{p} \in P : \|\vec{a} - \vec{p}\|_2 \leq r \implies (p, a) \in E \quad (4.3)$$

³<https://stanford.edu/~rqi/pointnet2/images/pnpp.jpg>

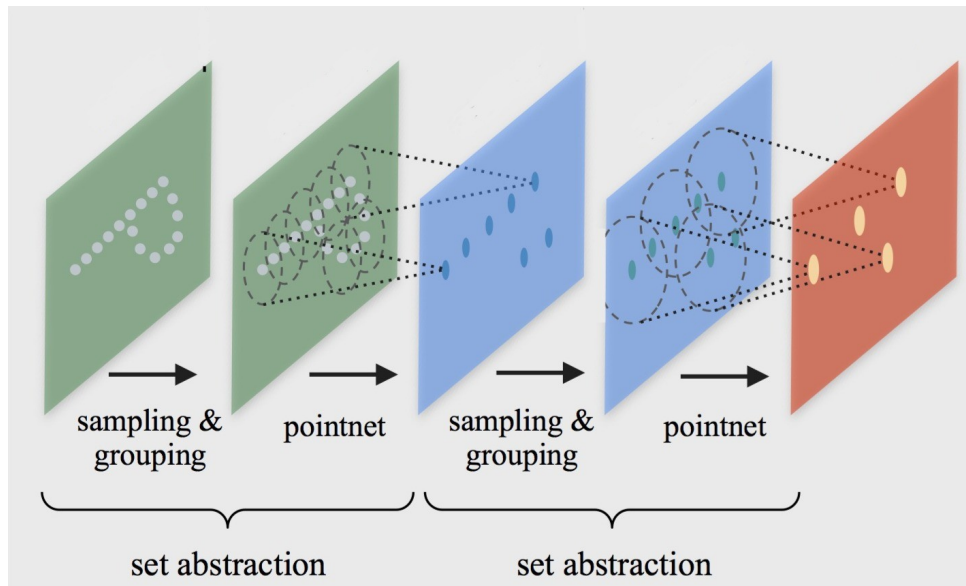


Abbildung 4.3: Schematische Darstellung von Merkmalsextraktionschichten³

Die ausgewählten Punkte werden als künstliche Neuronen genutzt. Dabei sind die Merkmale, die in den Punkten der Nachbarn gespeichert sind, die Eingabe für das künstliche Neuron. Die Gewichte für jede Eingabe werden erlernt, dafür wird ein Multi Layer Perceptor (MLP) genutzt. Der MLP lernt, wie die Gewichte für die oben bestimmten Kanten sein sollen. Dafür nutzt der MLP die relative Position von ausgewählten Punkten zu ihren Nachbarn. Ein MLP funktioniert wie ein voll verbundenes Netz, was in Abschnitt 4.2 erklärt wird. Ein ausgewählter Punkt verändert seine Merkmale anhand der Summe von Merkmalen der Punkte in seiner Nachbarschaft, die nach ihrer relativen Position mit dem MLP gewichtet werden. Die Anzahl an Punkten verringert sich, aber die Anzahl der Merkmale pro Punkt erhöht sich mit jeder Schicht.

Über mehrere Schichten hinweg, werden Merkmale für die Punkte gelernt, um dann die Punkte zu reduzieren und die Merkmale an die ausgewählten Punkte weiterzugeben. Das generiert eine Menge an Merkmalen für jeden Punkt, aber für die Weiterverarbeitung wird ein Merkmalsvektor benötigt, der das gesamte Bauteil beschreibt. Im letzten Schritt wird das größte Merkmal in jeder Dimension der Merkmale aller Punkte genommen. So wird ein flacher Merkmalsvektor erstellt. Mit dem größten Merkmal ist das numerisch Größte gemeint, nicht unbedingt das Wichtigste.

```

Eingabe: Punktwolke,  $n$ 
 $i = 0$ 
auswahl = Punktwolke.zufallPunkt()
while  $i < n$  do
    kandidat = Punktwolke.zufallPunkt()
    for  $punkt \in \text{Punktwolke} \setminus \text{auswahl}$  do
        minDistanz =  $\infty$ 
        for  $\text{auswahlPunkt} \in \text{auswahl}$  do
            if  $\text{Distanz}(\text{auswahlPunkt}, \text{Punkt}) < \text{minDistanz}$  then
                 $\text{minDistanz} = \text{Distanz}(\text{auswahlPunkt}, \text{Punkt})$ 
            end if
        end for
        if  $\text{minDistanz} > \text{kandidat.minDistanz}$  then
             $\text{kandidat} = \text{punkt}$ 
        end if
    end for
     $\text{auswahl} = \text{auswahl} + \text{kandidat}$ 
     $i = i + 1$ 
end while
RETURN  $\text{auswahl}$ 

```

Algorithmus 4.1: Farthest Point Sampling Algorithmus

4.2.3 Regression auf Merkmalsvektor

Nachdem für Punktwolken einer Fläche im Bauteil, mit dem in Abschnitt 4.2.2 beschriebenen Verfahren, die Merkmale der jeweiligen Punktwolken berechnet wurden, kann der Merkmalsvektor der jeweiligen Punktwolken zu der Z-Invarianz weiterverarbeitet werden. Dafür wird eine Abbildung von dem Merkmalsvektor auf die Z-Invarianz erlernt. Dieses Vorgehen wird Regression genannt. In diesem Fall wird auf die Z-Invarianz regressiert. Der Merkmalsvektor wird über mehrere Schichten auf eine Ausgabe abgeleitet.

Eine Schicht funktioniert so, dass jedes Neuron die gewichtete Summe von allen vorhergehenden Neuronen erhält und diese Summe an eine Aktivierungsfunktion übergibt. Dabei werden Methoden wie Normalisierung und zufälliges zurücksetzen von Eingaben verwendet, um die Transformation verlässlicher zu machen. Es sind voll verbundene Schichten, die in Abschnitt 4.2 beschrieben wurden. In jeder Schicht werden Maßnahmen wie Normalisierung und zufälliges zurücksetzen eingesetzt, um die Regression stabiler zu machen.

Die Normalisierung stellt sicher, dass keine zu großen Änderungen innerhalb einer Schicht vorkommen, da alle Werte relativ zu dem Durchschnitt normalisiert werden.

Bei einer großen Änderung innerhalb einer Schicht, zwingt es darauffolgende Schichten ebenfalls Änderungen vorzunehmen. Die konstante Anpassung zu Änderungen in den vorherigen Schichten macht das Lernen instabil.

Das zufällige Zurücksetzen von Eingaben verhindert eine zu große Abhängigkeit von nur einem Neuron oder einem Pfad. Dafür werden mit einer vorbestimmten Wahrscheinlichkeit gewichtete Summen auf null gesetzt, sodass sie keinen Einfluss auf die nächste Schicht haben.

In jeder Schicht des Regressionsnetzes wird dieselbe Aktivierungsfunktion genutzt. In allen Schichten außer der letzten ist das $f(x) = \max(0, x)$ die als Rectifier oder ReLU bekannt ist. Es ist eine der bekanntesten Aktivierungsfunktionen, unter anderem weil sie einfach, aber effektiv ist. Für die letzte Aktivierung wird dagegen

$$f(x) = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)} \quad (4.4)$$

genutzt. Diese Funktion wird tanh genannt. Sie hat den Vorteil, dass ihre Ausgabe immer zwischen -1 und 1 liegt, was genau das Intervall an Werten ist, die in einer Rotationsmatrix zugelassen sind. Die Ausgabe muss also nicht korrigiert werden, um für eine Rotationsmatrix zulässig zu sein.

Das Ergebnis der Regression ist die Z-Invarianz. Die Z-Invarianz ist aber noch keine Rotationsmatrix und muss daher zu einer umgewandelt werden, um ein Bauteil drehen zu können. Dies wird in Abschnitt ?? beschrieben.

4.3 Konstruktion der Rotationsmatrix

Die, in Abschnitt 4.2 berechnete, Z-Invarianz ist nicht genug, um eine Drehung darzustellen. Um ein Bauteil drehen zu können ist eine Rotationsmatrix notwendig. Die Z-Invarianz in diesem Abschnitt wird zu einer Rotationsmatrix erweitert. Rotationsmatrizen sind, für den 3d Raum, 3 x 3 Matrizen, die durch Matrixmultiplikation eine Punktwolke drehen können. Eine Punktwolke bestehend aus mehreren Punkten (x_i, y_i, z_i) kann wie folgt durch Matrixmultiplikation mit einer Rotationsmatrix R gedreht werden.

$$\begin{bmatrix} x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \\ \vdots & \vdots & \vdots \\ x_n & y_n & z_n \end{bmatrix} \begin{bmatrix} R_{11} & R_{12} & R_{13} \\ R_{21} & R_{22} & R_{23} \\ R_{31} & R_{32} & R_{33} \end{bmatrix} = \begin{bmatrix} x'_1 & y'_1 & z'_1 \\ x'_2 & y'_2 & z'_2 \\ \vdots & \vdots & \vdots \\ x'_n & y'_n & z'_n \end{bmatrix} \quad (4.5)$$

Dabei ist die Norm von jeder Zeile gleich Eins und jede Zeile ist, wenn man sie als Vektor betrachtet orthogonal zu den anderen Zeilen. Dasselbe gilt für Spalten.

$$\forall i \in \{1, 2, 3\} : \left\| \begin{bmatrix} R_{1i} \\ R_{2i} \\ R_{3i} \end{bmatrix} \right\|_2 = 1 \quad (4.6)$$

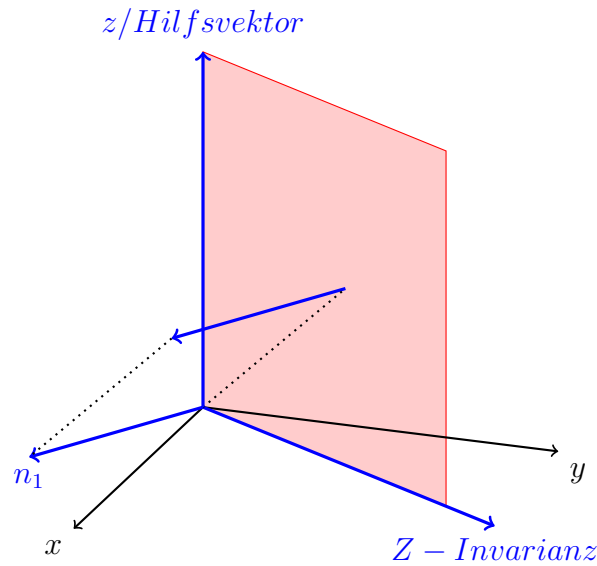
$$\forall i \in \{1, 2, 3\} : \left\| \begin{bmatrix} R_{i1} \\ R_{i2} \\ R_{i3} \end{bmatrix} \right\|_2 = 1 \quad (4.7)$$

Um aus dem 3d Vektor, der die Z-Invarianz darstellt, eine Rotationsmatrix zu erstellen, kann die Z-Invarianz selber als Zeile der Rotationsmatrix genommen werden. Die restlichen Zeilen müssen dann aus der ersten Zeile abgeleitet werden. Die Zeilen einer Rotationsmatrix müssen Orthogonal zueinander sein, daher muss für die anderen Zeilen Vektoren gefunden werden die Orthogonal zu der Z-Invarianz sind. Wenn zwei Vektoren gegeben sind, kann man aus ihnen eine Ebene bilden. Ein Normalvektor dieser Ebene ist Orthogonal zu beiden Vektoren. Dadurch lassen sich orthogonale Vektoren zu der Z-Invarianz finden.

Um eine Ebene zu erstellen, benötigt es mindestens zwei Vektoren. Die Z-Invarianz ist der Erste. Als zweiter wird die z-Achse, als Hilfsvektor, genommen. Als Hilfsvektor kann jeder Vektor genommen werden, solange er die Länge eins hat und sich nicht ändert. Die z-Achse wurde gewählt, damit sich die von der Z-Invarianz beschriebene Rotation auf die z-Achse bezieht. Wenn man die x-Achse als Hilfsvektor nimmt, würde sich die Ebene parallel zu der x-Achse ausrichten.

In der Abbildung 4.4 wird die Berechnung von der Normale-1 (n_1) dargestellt. Die aus der Z-Invarianz und dem Hilfsvektor geformte Ebene ist rot gefärbt. Die Z-Invarianz, der Hilfsvektor und n_1 sind blau gefärbt. n_1 steht senkrecht zu der Ebene und ist daher der Normalvektor der Ebene. Der Normalvektor einer Ebene kann aus dem Kreuzprodukt von zwei nicht gleichen Vektoren die auf einer Ebene liegen berechnet werden. In diesem Fall sind die Z-Invarianz und die z-Achse, als Hilfsvektor, diese zwei Vektoren. Da die Z-Invarianz aber nur ein Vektor ist, muss für die Bestimmung anderer Zeilen im ersten Schritt ein Hilfsvektor genommen werden. Dafür bietet sich die z-Achse an, da die Rotationen relativ zu der z-Achse bewertet werden. Zwischen der Z-Invarianz und der z-Achse wird eine Ebene erstellt. Der Normalvektor der Ebene ist eine Zeile in der Rotationsmatrix.

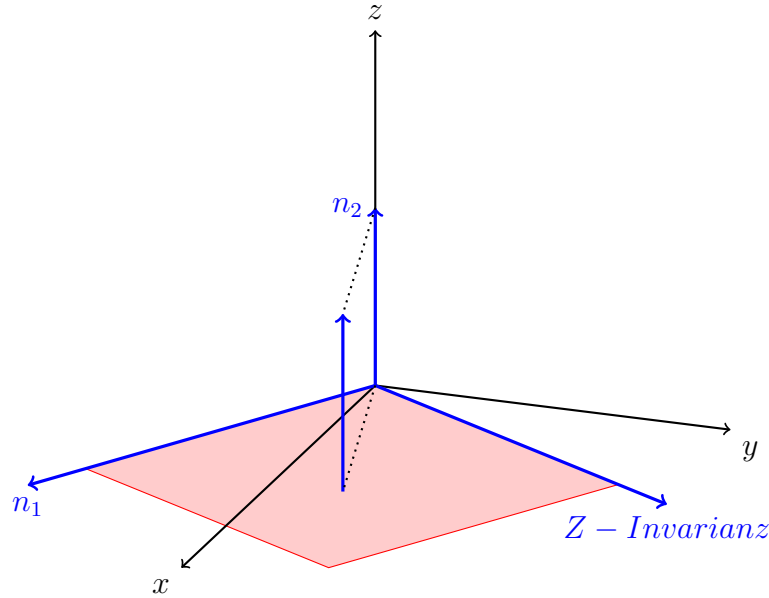
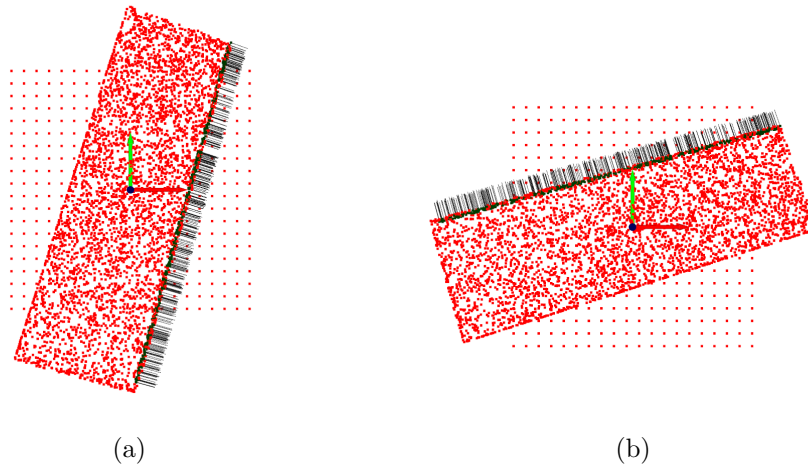
Als Hilfsvektor wird immer die normierte z-Achse genommen. Daher besteht die

Abbildung 4.4: Berechnung von n_1

Möglichkeit, dass die Z-Invarianz sehr ähnlich zu der z-Achse ist. Das macht die Berechnung des Normalvektors mathematisch instabil, da durch die extreme Ähnlichkeit jetzt der Nullvektor als Ergebnis kommen könnte. Um dieses Problem zu beheben, wird überprüft, ob der Hilfsvektor zu ähnlich zu der Z-Invarianz ist. Wenn er zu ähnlich ist, wird der Hilfsvektor durch einen anderen Vektor ersetzt. Einen anderen Hilfsvektor zu nehmen beeinflusst die Rotation, aber diese Ausnahme ist sehr selten und verschlechtert das durchschnittliche Ergebnis nicht sehr. Als Alternative zu der z-Achse wird ein Vektor, der nahe an der z-Achse liegt genommen.

In der Abbildung 4.5 wird die Berechnung von der Normale-2 (n_2) dargestellt. Die erstellte Ebene zwischen n_1 und der Z-Invarianz ist rot gefärbt. Senkrecht zu der roten Ebene ist, in blau, n_2 zu sehen. n_2 ist der Normalvektor der Ebene. Durch das Berechnen einer anderen Zeile außer der Z-Invarianz ist ein Hilfsvektor nicht mehr notwendig. Die berechnete Normale n_2 bildet die letzte Zeile, die für eine Rotationsmatrix vonnöten ist. Sie ist orthogonal zu den anderen Zeilen, da sie eine Normale in der Ebene der anderen Zeilen ist. Normale-1 ist ebenfalls orthogonal zu der Normale-2, da es in der Ebene ist, dessen normale Normale-2 ist. Sie ist ebenfalls orthogonal zu der Z-Invarianz, da Normale-1 aus Ebene, die die Z-Invarianz beinhaltet gebildet wurde. Orthogonalität ist symmetrisch, daher ist auch die Z-Invarianz orthogonal zu Normale-1 und Normale-2. Das erfüllt die oben genannte Bedingung, dass alle Zeilen orthogonal zueinander sein müssen.

In der Abbildung 4.6 ist der Unterschied, den das Vertauschen von n_1 und n_2 machen kann, dargestellt. In Abbildung 4.6(a) und 4.6(b) ist dasselbe Bauteil zu sehen. Die

Abbildung 4.5: Berechnung von n_2 Abbildung 4.6: Vertauschen von n_1 und n_2 in der Rotationsmatrix

Punkte, die das Bauteil markieren, sind rot. Die Fläche, die rotiert wird, ist in Abbildung 4.6(a) auf der rechten Seite und in Abbildung 4.6(b) auf der oberen Seite zu sehen. Aus den Punkten der Fläche kommen in beiden Fällen schwarze Stiche. Sie zeigen in Richtung des Normalvektors der Fläche. Die Z-Invarianz wird als unterste Zeile genommen, die beiden berechneten Normalen können eine beliebige andere Zeile nehmen. Die Wahl der Zeilen verändert nur die Rotation des Bauteils um die z -Achse herum, wie man in der Abbildung 4.6 erkennen kann. Eine Drehung um die z -Achse verändert die Gradabweichung von der z -Achse nicht. Es ist daher für die Ergebnisse egal wie n_1 und n_2 in der Rotationsmatrix angeordnet werden.

Um für vergleichbare Ergebnisse zu sorgen, sollten n_1 und n_2 dennoch immer gleich angeordnet werden.

Nachdem die Rotationsmatrix erstellt wurde muss zuletzt überprüft werden, ob sie eine gültige Rotation ist. Im 5d-Druck kann sich die Plattform nicht völlig frei bewegen. Sie kann eine maximale Gradabweichung vom Ursprung von 90 Grad unterstützen, also insgesamt jeweils 180 Grad entlang zwei Achsen. Daher wird überprüft, ob die Rotationsmatrix das Bauteil über diese Grenzen hinaus drehen würde. Falls dem so ist, muss die Rotationsmatrix gespiegelt werden.

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & -1 \end{bmatrix} \quad (4.8)$$

In der Formel 4.8 wird die Spiegelmatrix gezeigt. In der Abbildung 4.7 wird die Veränderung, die sie auf eine Rotationsmatrix wirkt, dargestellt. Die Linke Abbildung 4.7(a) zeigt eine unveränderte Rotationsmatrix, die rechte Abbildung 4.7(b) zeigt eine Rotationsmatrix, die mit der Spiegelmatrix in 4.8 verändert wurde. Durch die Matrixmultiplikation mit der Spiegelmatrix, aus 4.8, wird eine Rotationsmatrix in der XY-Ebene gespiegelt. Das hat den Effekt, dass sich das Bauteil um 180 Grad dreht. Da eine Spiegelung nötig war, galt vor der Spiegelung, dass der Wert der Rotation mehr als 90 Grad ist. Die Rotation ist also zwischen 90 und 180 Grad. Sie kann nicht höher sein, da eine sinnvolle Rotation nur von -180 bis 180 Grad gehen kann und so einen ganzen Kreis bildet. Wenn man die Rotation spiegelt, wird sie um 180 Grad gedreht.

Wenn sie vor der Spiegelung nicht im 180 Grad Halbkreis der erlaubten Drehungen war, dann ist sie es danach. In dieser Arbeit wird von einem 90 Grad Maximum in der Drehung der Plattform in beide Dimensionen ausgegangen. In der Realität ist es möglich, dass ein Drucker zu weniger in der Lage ist. Aber 90 Grad ist die höchste sinnvolle Schwenkung der Plattform, bei der die Plattform nicht die Düse, des Druckers blockieren kann. Eine Plattform, die weniger als 90 Grad schwenken kann, kann viele der berechneten Ausrichtungen nicht umsetzen. Dieses Szenario wird in dieser Arbeit nicht betrachtet, da nur bewertet werden soll wie praktikabel der Ansatz ist, Hardwareeinschränkungen sind dafür zuerst zu vernachlässigen.

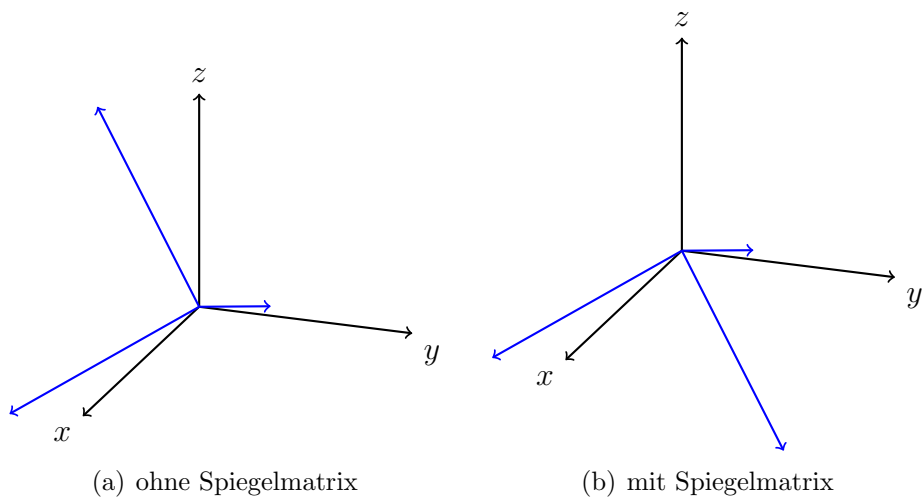


Abbildung 4.7: Veränderung der Rotationmatrix durch die Spiegelmatrix

5 Evaluation

5.1 Versuchsaufbau

Die Ausrichtung eines Bauteils ist in den meisten Fällen nicht perfekt. In diesem Kapitel wird behandelt wie gut das in Kapitel 4 erstellte neurales Netz ein Bauteil ausrichten kann. Dafür wird nicht nur die Ausrichtung selber überprüft. Es wird auch die Größe der Rotationen untersucht, da das Drehen eines Bauteils Zeit kostet und die Minimierung dieser Drehung wie in 2.3 erwähnt eine Priorität ist. Außerdem wird die Fähigkeit des neuralen Netzes Fehler in den Daten zu widerstehen in Abschnitt 5.3 überprüft und die Fähigkeit auf komplexen Bauteilen zu arbeiten in Abschnitt 5.4 getestet.

Alle Daten, die für die Auswertung benutzt werden, sind in dem Artikel: [Zel+25] erstellt worden und dann wie in Kapitel 3 beschrieben verändert worden. Die Daten sind künstlich generiert worden und dementsprechend nicht echte Bauteile, sondern einfache geometrische Objekte, denen teilweise Löcher oder anderen Eigenschaften gegeben wurden. Die Abbildung 5.1 stellt alle Typen von Bauteilen dar, die in dem Datensatz enthalten sind. Es gibt insgesamt 4 verschiedene Typen, die alle einfache geometrische Objekt sind.

Die Typen sind von A bis D gekennzeichnet. Die unterschiedlichen Typen sind: "PP-latten mit Löchern", "Platonischer Körper mit Inschriften", "Körper mit mehreren Löchern und Pyramiden".

Für alle Tests, in dem Abschnitt 5.3.1 und 5.3.2, werden 1000 zufällige Flächen aus dem Datensatz genommen. Diese Flächen werden wie in Kapitel 4 beschrieben gefunden. Tests haben ergeben, dass das Verwenden von einem Datensatz mit mehr als 1000 Flächen keinen signifikanten Einfluss auf das Ergebnis hat.

¹https://media.springernature.com/full/springer-static/image/art%3A10.1007%2Fs40964-025-00960-6/MediaObjects/40964_2025_960_Fig1_HTML.png

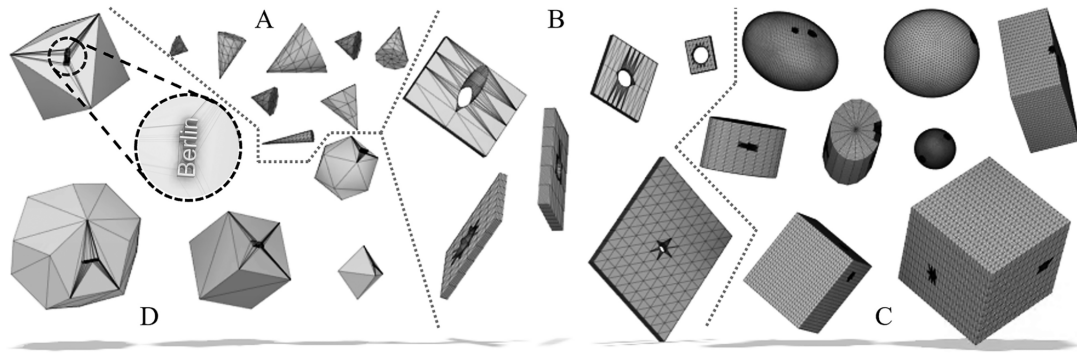


Abbildung 5.1: Darstellung aller Typen von Bauteilen im Datensatz¹

5.2 Ausrichtung der Bauteile

Die Ausrichtung der Bauteile ist das Ziel der Arbeit. Dementsprechend ist es die wichtigste Metrik für den Erfolg des neuronalen Netzes. Die Minimierung der Bewegung ist ebenfalls wichtig, da jede Drehung eines Bauteils Zeit kostet. Als zweites Ziel des Netzes wird dementsprechend versucht, die Größe der Drehung des Bauteils zu minimieren.

Bauteil	Gradabweichung vom Optimum	Bewegung des Bauteils
Platten mit Löchern	10.12	23.56
Platonischer Körper	9.59	26.50
Körper mit zwei Löchern	9.40	27.11
Pyramiden	9.22	24.68

Tabelle 5.1: Testergebnisse auf einfachen Bauteilen

Die Tabelle 5.1 zeigt die Ergebnisse der Tests auf den einfachen Bauteilen des Datensatzes. Dabei ist jeder Typ von Bauteil einzeln aufgeführt. Es lässt sich erkennen, dass alle Bauteile, außer die Platten mit Löchern, eine Genauigkeit von unter 10 Grad Abweichung vom Optimum erreichen. Die Platten mit Löchern erreichen eine Genauigkeit von 10.12 Grad Abweichung vom Optimum, was nicht viel höher ist. Insgesamt lässt sich kein großer Unterschied zwischen den verschiedenen Typen von Bauteilen erkennen. Alle Typen erreichen eine ähnliche Genauigkeit und benötigen eine ähnliche Bewegung des Bauteils. Die Pyramiden erreichen die beste Genauigkeit mit einer Abweichung von 9.22 Grad vom Optimum und einer durchschnittlichen Bewegung von 24.68 Grad. Die Pyramiden sind die einfachsten Bauteile im Datensatz, was die gute Genauigkeit erklären könnte. Andererseits sind die Platten mit Löchern die am schlechtesten abschneidenden Bauteile, was die Gradabweichung

vom Optimum angeht. Das könnte daran liegen, dass die Flächensuche Probleme mit den Löchern in den Platten hat.

Der vorherige Test 5.1 wurde durchgeführt, sodass sowohl auf die Minimierung der Bewegung als auch auf die optimale Ausrichtung geachtet wird. Es ist allerdings auch relevant, wie gut das Netz eine Fläche ausrichten kann, wenn einen die Größe der Bewegungen vernachlässigt.

Bauteil	Gradabweichung vom Optimum	Bewegung des Bauteils
Platten mit Löchern	5.42	59.19
Platonischer Körper	5.67	58.28
Körper mit zwei Löchern	4.85	61.02
Pyramiden	4.26	58.79

Tabelle 5.2: Testergebnisse auf einfachen Bauteilen ohne Berücksichtigung von der Bewegung des Bauteils

Es lassen sich starke Verbesserungen in der Tabelle 5.2, was die Ausrichtung angeht, erkennen, wenn man es mit der Tabelle 5.1 vergleicht. Die Ergebnisse haben sich durch das Vernachlässigen der Bewegung des Bauteils halbiert und haben jetzt eine Abweichung von in etwa 5 Grad. Die Platten mit Löchern sind immer noch der am schlechtesten abschneidende Typ von Bauteil und die Pyramiden sind immer noch die am besten Abschneidenden. Dafür, dass sich die Abweichung vom Optimum halbiert hat, hat sich die Bewegung des Bauteils mehr als verdoppelt. Da sie von dem Netz für diesen Test nicht beachtet wurde, war diese Veränderung zu erwarten.

5.3 Verlässlichkeit der Ausrichtung

Ein wichtiger Unterschied zwischen dem Trainingsdatensatz und der Praxis ist, dass man in der Praxis nicht davon ausgehen kann, dass die Daten einwandfrei sind. Um ein praxisrelevantes Verfahren zu entwickeln, muss das Verfahren in der Lage sein, Fehler in den Daten zu widerstehen. In dem bisherigen Test ist immer davon ausgegangen worden, dass die Punktwolken eine perfekte Darstellung eines Bauteils ist. In der Praxis, muss aber davon ausgegangen werden, dass die Punktwolken Fehler aufweisen. Um diese Fehler zu simulieren, wird in den folgenden zwei Abschnitten der Einfluss von fehlenden Punkten und der Einfluss von falschen Punkten untersucht.

5.3.1 Effekt von Rauschen auf den Daten

Eine Fläche eines Bauteils, die Punkte beinhaltet, die nicht zu der Fläche gehören, kann das Ergebnis des neuronalen Netzes stark beeinflussen. Dieser Fehler könnte durch schlechtes Clustering entstehen, wenn zwei Flächen nicht richtig getrennt werden. Punkte die nicht zu der Fläche gehören, aber dennoch in der Punktwolke der Fläche enthalten sind, werden als Rauschen (engl. Noise) bezeichnet. Um den Einfluss von Rauschen auf das Netz zu bestimmen, werden Punkte zu sonst korrekten Flächen hinzugefügt.

Die Abbildung 5.2 zeigt die Ergebnisse des Tests. Um Noise zu simulieren, wurden für jede Fläche um das Zentrum der Fläche zufällig Punkte verteilt. Dabei wurden die Punkte um das Zentrum der Fläche normalverteilt, sodass die meisten Punkte nahe der Fläche sind. Die zufällige Verteilung bricht die kohärente Struktur der Fläche auf. In der Abbildung 5.2 wurde der Unterschied des Normalvektors der Fläche zu der z-Achse in Orange aufgezeichnet, die Standardabweichung des Normalvektors zu der z-Achse in Grün und die benötigte Größe der Drehung ist in Blau. Die x-Achse der Abbildung zeigt das Rauschen. Dabei ist die Anzahl der Rauschpunkte gleich des Wertes auf der x-Achse mal 20. Die y-Achse zeigt die Ergebnisse bei den jeweiligen Rauschleveln. Abbildung 5.2 zeigt, dass das Rauschen keinen großen Einfluss auf den Unterschied des Normalvektors der Fläche zu der z-Achse oder die benötigte Größe der Drehung hat. Auf die Standardabweichung des Normalvektors hat das Rauschen allerdings einen extremen Einfluss. Sobald Rauschen hinzugefügt wird, steigt die Standardabweichung extrem an und bleibt dann bei dem Fehlermaß, zu dem es steigt, unabhängig von weiterem Rauschen, dass hinzugefügt wird. Daraus lässt sich schließen, dass das Netz keine gute Fähigkeit hat, Rauschen zu widerstehen. Die absolute Ausrichtung in Orange bleibt zwar stabil, aber da die Abweichung von diesem Durchschnitt extrem ansteigt, ist das Ergebnis unzuverlässig.

Der extreme Anstieg, der Standardabweichung in Abbildung 5.2 könnte dadurch erklärt werden, dass das Netz nur auf fehlerfreien Daten trainiert wurde. Die Rauschpunkte eliminieren die glatte Struktur der Fläche, aber die Merkmalsextraktion wird dennoch versuchen diese Punkte mit einzubeziehen und dadurch auf falsche Merkmale kommen. Das erklärt ebenfalls, warum die Standardabweichung nach dem ersten Anstieg konstant bleibt. Sobald die Merkmale falsch sind, ändert weiteres Rauschen nichts mehr an den falschen Merkmalen. Die Ergebnisse sind dennoch besser als Zufällig, was darauf hindeutet, dass das Netz trotz des Rauschens noch die Originale Fläche erkennt, aber die Rauschpunkte eine große Unsicherheit in die Ergebnisse bringen.

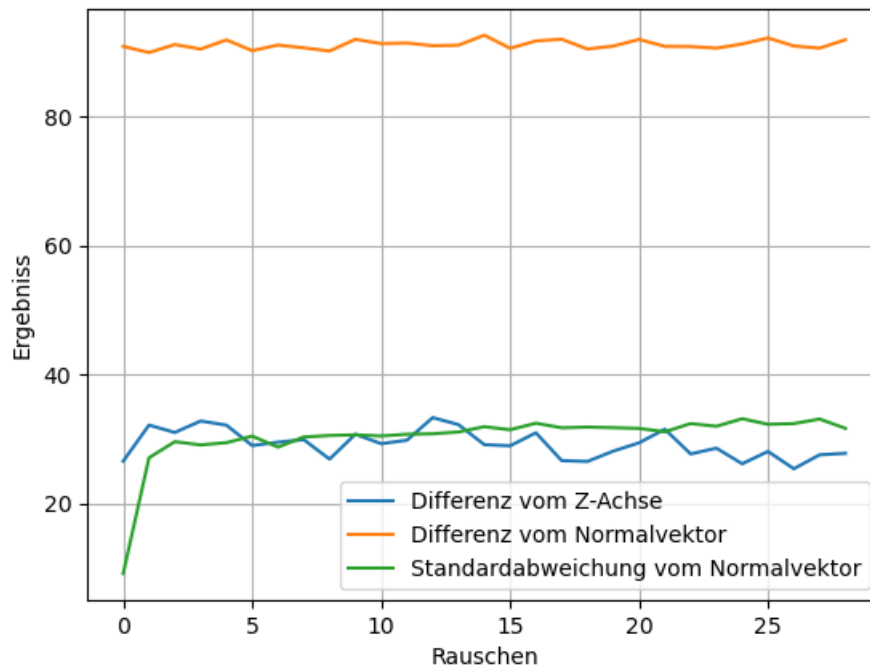


Abbildung 5.2: Test mit Noise auf dem Neuralen Netz

5.3.2 Effekt von schlechter Flächensuche

Eine Fläche, die in einem Bauteil gefunden wurde, muss nicht immer perfekt die Fläche des Bauteils repräsentieren. Häufig werden Punkte, die zu der Fläche gehören sollten, nicht gefunden, was die Punktwolke kleiner macht als sie es mit einer perfekten Suche wäre. Der Einfluss von fehlenden Punkten in der Punktwolke einer Fläche wird getestet, indem über mehrere Iterationen Punkte aus den Punktwolken entfernt werden. Die Anzahl der Punkte basiert auf einem Prozentsatz, der sich mit jeder Iteration erhöht. Durch das Verwenden von Prozentsätzen wird sichergestellt, dass die Ergebnisse unabhängig von der ursprünglichen Größe der Punktwolke sind. Die Abbildung 5.3 zeigt den Verlust an Genauigkeit des Netzes, wenn Punkte aus der Punktwolke entfernt werden. Alle Ergebnisse sind in Grad angegeben. Auf der x-Achse ist der Prozentsatz der entfernten Punkte dargestellt, auf der y-Achse ist das Ergebnis der Tests dargestellt. Die Abweichung von dem Normalvektor der Fläche ist nahezu unverändert, wenn Punkte entfernt werden. Die Standardabweichung des Normalvektors bleibt für die ersten 80 %, ebenfalls fast unverändert, und steigt nur leicht an. Erst nachdem 80 % der Punkte entfernt wurden, steigt die Standardabweichung stark an. Bei 90 % entfernten Punkten ist die Standardabweichung fast

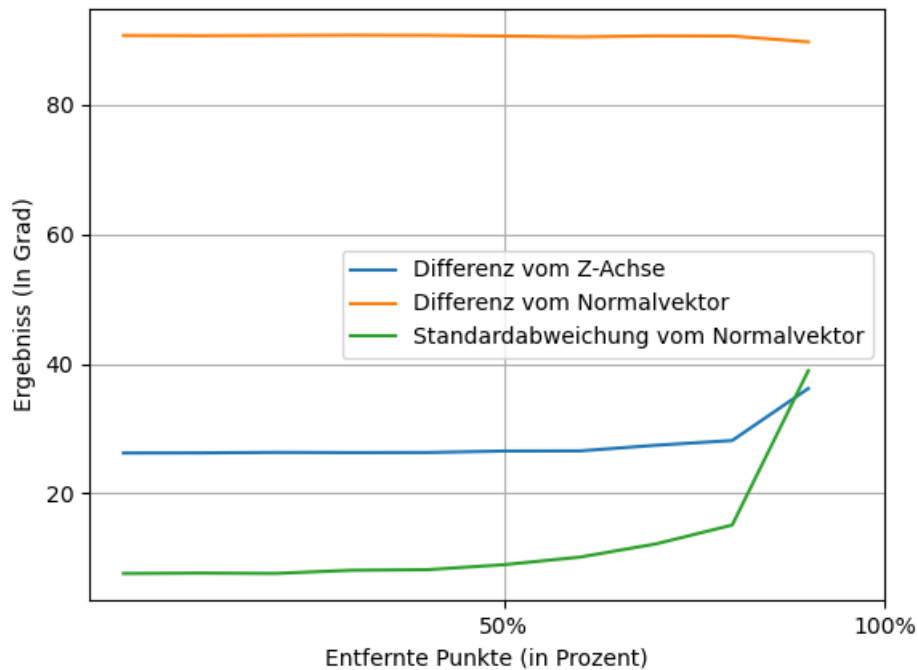


Abbildung 5.3: Test von schlechter Flächensuche auf dem Neuralen Netz

bei 45 Grad angekommen, was bedeutet, dass bei einem so hohen Verlust das Ergebnis kaum besser als Zufall ist. Aber bis zu diesem Punkt zeigt das Netz eine gute Fähigkeit, fehlende Punkte in der Punktwolke zu widerstehen. Ein Verlust von über 80 % der Punkte ist extrem unwahrscheinlich in der Praxis. Das Netz zeigt also eine gute Widerstandsfähigkeit gegen schlechte Flächensuche.

Auch bei der Differenz zu der z-Achse oder der Bewegung des Bauteils ist kein großer Einfluss von fehlenden Punkten zu erkennen, bis 80 % der Punkte entfernt wurden. Danach hat auch hier der Verlust an Punkten einen großen Einfluss auf die Genauigkeit des Netzes, allerdings ist der Anstieg deutlich geringer im Vergleich zu der Standardabweichung. Der Punkt, an dem die Genauigkeit stark abfällt, ist bei der Standardabweichung und der Differenz zu der z-Achse gleich. Das deutet darauf hin, dass der Abfall der Genauigkeit denselben Grund hat.

Der Zerfall der Genauigkeit in Abbildung 5.3 könnte dadurch erklärt werden, dass die Merkmalsextraktion darauf basiert, dass eine Menge an Punkten Merkmale besitzt, die in jedem Schritt auf eine kleinere Menge an Punkten reduziert werden. Wenn zu viele Punkte fehlen, dann wird das Netz weniger Punkte für die Reduktionen haben, was den langsamen Zerfall der Standardabweichung bis zu der 80 % Grenze erklären könnte. Warum es danach so stark abfällt, ist vermutlich damit zu erklären, dass

die Dichte der Punkte so stark abgenommen hat, dass die Reduktion häufig keine Punkte in der lokalen Umgebung hat, die sie für eine Reduzierung verwenden kann. Wenn die Merkmale von einem oder sehr wenigen Punkten aus reduziert werden, denn zerfällt die Idee, lokale Merkmale anhand Ihrer relativen Position zu andern Punkten zu bewerten. Das neurale Netz verliert die Fähigkeit, lokale Merkmale zu erkennen.

5.4 Generalisierbarkeit

Alle bisherigen Auswertungen wurden auf den gleichen Typen von Daten durchgeführt, auf denen das Netz auch trainiert wurde. Um die Verlässlichkeit des Netzes zu überprüfen wird in diesem Abschnitt getestet, ob das Netz auch auf komplexeren Bauteilen arbeiten kann. Für diesen Zweck wurden mehrere komplexe Bauteile getestet. Komplex bedeutet in diesem Fall, dass die Bauteile nicht aus einfachen geometrischen Formen bestehen, sondern gebogene Flächen und komplexe Strukturen beinhalten.

Die Abbildung 5.4 sind die 3d-Modelle aller komplexen Bauteilen abgebildet. Die Abbildung A zeigt den Stanford Hasen (engl. Stanford Bunny). Er ist asymmetrisch und hat an den Ohren dünne, gekrümmte Flächen. Der gesamte Körper ist mit Fell bedeckt und dadurch wird eine schon gebogene Fläche mit mehreren Unebenheiten überdeckt. Der Stanford Drache ist ebenfalls asymmetrisch. Seine Oberfläche ist gebogen und ist durch viele Spitzen auf der Oberfläche nicht glatt. Er hat ebenfalls mehrere Stellen, wie der Kopf oder die Mitte des Körpers, an denen der Körper keine nahegelegene Stütze vom Körper selber hat. Zuletzt ist der Utah Teepott (engl. Utah Teapot) das einzige komplexe Bauteil, was zum größten Teil symmetrisch ist. Der Teepott hat eine glatte Oberfläche, die häufig nicht gekrümmt ist. Die Bauteile eignen sich als Beispiel für komplexe Bauteile, da sie meistens aus unebenen und gebogenen Flächen bestehen. Der Teepott ist weniger komplex, aber immer noch komplexer als die einfachen Daten in dem Abschnitt 5.2. Es ist ein Alltagsgegenstand und kann als Vergleichswert dienen.

Alle Angaben in der Tabelle 5.3 sind Durchschnittswerte über alle Flächen eines Bauteils. Die Ergebnisse sind immer in Grad und für jedes Bauteil einzeln angegeben. Es lässt sich erkennen, dass die Abweichung vom Optimum bei komplexen Bauteilen höher ist, als bei den einfachen Bauteilen, die in Abschnitt 5.2 getestet

⁴<https://graphics.stanford.edu/data/3Dscanrep/stanford-bunny-cebal-ssh.jpg>

⁴<https://graphics.stanford.edu/data/3Dscanrep/dragon.jpg>

⁴<https://images.computerhistory.org/revonline/images/x398.84p-03-02.jpg?w=600>

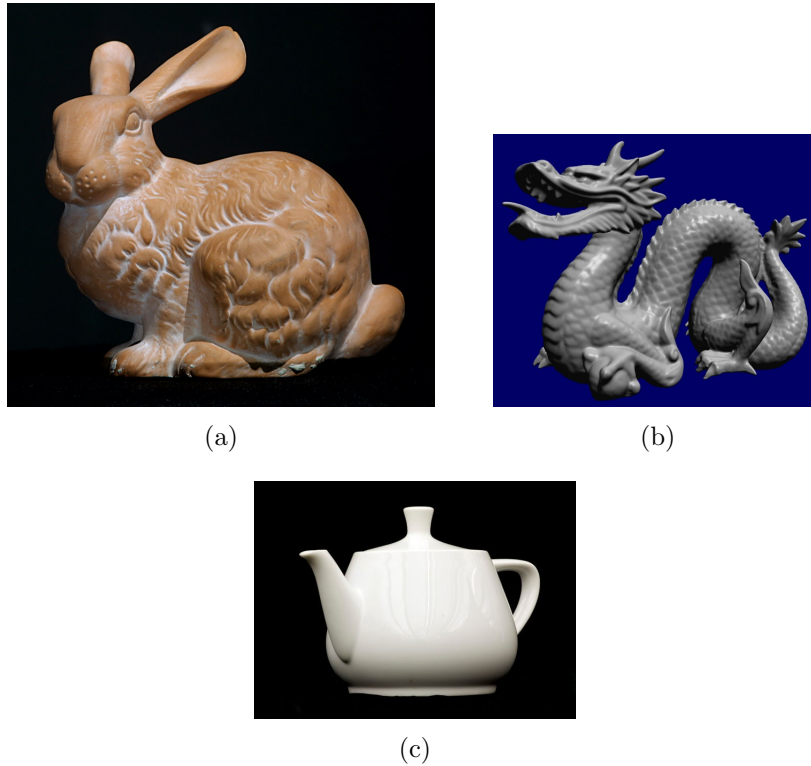


Abbildung 5.4: 3d-Modelle der komplexen Bauteile²³⁴

Bauteil	Gradabweichung vom Optimum	Bewegung des Bauteils
Stanford Hase	10.76	17.29
Stanford Drache	11.61	25.02
Utah Teapot	10.84	26.72

Tabelle 5.3: Testergebnisse auf komplexen Bauteilen

wurden. Die Abweichung vom Optimum liegt bei allen komplexen Bauteilen bei ungefähr 11 Grad. Der geringe Unterschied zwischen den Ergebnissen der komplexen Bauteile lässt darauf schließen, dass die Auswahl der komplexen Bauteile eine gute Repräsentation für komplexe Bauteile im Allgemeinen ist. Allerdings ist die Bewegung des Bauteils bei dem Stanford Hasen deutlich geringer als bei den anderen beiden Bauteilen. Der Unterschied könnte daran liegen, dass der Stanford Hase nur eine Grundfläche hat. Grundflächen werden immer für hohe Bewegung sorgen, da die optimale Ausrichtung meistens im 90 Grad Winkel zu der Grundfläche liegt. Der Stanford Drache und der Utah Teapot haben beide mehrere Grundflächen, was die Bewegung des Bauteils erhöhen könnte.

Der Teepott hat keine signifikant anderen Ergebnisse, im Vergleich zu den ande-

ren zwei Bauteilen, obwohl es eine deutlich einfacheren Form hat. Eine nach einer genaueren Untersuchung der Ausrichtung kann dieses Verhalten dadurch erklärt werden, dass die Flächensuche keine größeren Probleme mit der unebenen Fläche des Stanford Drachen oder Hasen hat. Auf unebenen Flächen werden immer noch Flächen ähnlicher Qualität wie auf glatten Flächen gefunden, sie sind aber meistens kleiner.

6 Zusammenfassung und Ausblick

6.1 Zusammenfassung

In dieser Arbeit wurde zunächst die Notwendigkeit der Ausrichtung von Bauteilen in der additiven Fertigung erläutert. Dabei wurde ganz besonders auf die zusätzlichen Herausforderungen bei dem 5D Druck eingegangen, in dem die Ausrichtung eine größere Rolle spielt, da hier die Ausrichtung eines Bauteils mehrmals während des Druckens geändert werden kann. Wobei eine Ausrichtung für jede Fläche des Bauteils zu berechnen sich als extrem komplex herausgestellt hat.

Anschließend wurde ein neuronales Netz als Lösungsansatz vorgestellt, welches in der Lage ist, die Ausrichtung eines Bauteils für Flächen in dem Bauteil zu bestimmen. Dafür wurde ebenfalls ein Verfahren zur Extraktion von Flächen aus Bauteilen beschrieben. Die Frage, die in der Arbeit gestellt wurde, war es, ob ein tiefes neuronales Netz in der Lage ist, eine praxisrelevante Lösung für das Problem der Teilausrichtung im 5d-Druck bietet.

Schließlich wurden Test auf dem beschriebenen neuronalen Netz durchgeführt und ihre Ergebnisse wurden ausgewertet. Daraus ergab sich, dass das Netz in der Lage ist, Bauteile mit einer akzeptablen, aber bei weitem nicht perfekten, Genauigkeit auszurichten. Eine Abweichung von unter 10 Grad ist im Durchschnitt erreicht worden. Bei einer Abweichung von bis zu 40 Grad ist das Drucken immer noch möglich, wenn auch nicht sehr schnell [Jia+18]. Dementsprechend ist eine solche Genauigkeit nicht optimal, aber dennoch ausreichend, um einen Überhang zu vermeiden. Auch auf komplexeren Bauteilen konnte das Netz verlässlich, akzeptable Ergebnisse erzielen, was auf eine gute Generalisierbarkeit des Netzes hinweist. Eine durchschnittliche Abweichung von 10 Grad ist für den Praxisfall häufig zu ungenau. 10 Grad reichen nicht aus, um einer Fläche einen Überhang zu geben, der Stützstrukturen notwendig machen würde. Die Ausrichtungen sind also gut genug, um verlässlich Bauteile ohne Stützstrukturen zu drucken. Das Ergebnis, wenn die Minimierung der Bewegung ignoriert wird, ist besser mit 5 Grad. Es würde allerdings auch eine signifikante Erhöhung der Druckzeit mit sich bringen, da sich die Bewegung für jede Ausrichtung

mehr als verdoppelt hat. Die Ausrichtung ist in beiden Fällen nicht perfekt, für eine praktische Anwendung würde man bessere Ergebnisse wollen. Die neurale Netz sind dennoch in der Lage eine Ausrichtung mit akzeptabler Abweichung zu finden. Es gibt außerdem keine Funktion im neuronalen Netz, die einen sauberen Übergang zwischen zwei Flächen garantiert. Der Unterschied im Druckwinkel zwischen zwei Schichten könnte dementsprechend häufig 20 Grad übersteigen. Der extreme Unterschied in Druckwinkel könnte strukturelle Probleme an den Übergangspunkten zweier Flächen verursachen.

Die Minimierung der Bewegung des Bauteils während des Ausrichtens hatte eine kleine, aber sichtbaren Verkleinerung der benötigten Drehung ergeben. Der durchschnittliche Winkel, um den ein Teil gedreht werden muss, lag bei unter 40 Grad. Ohne die Minimierung würde der durchschnittliche Drehwinkel bei über 45 Grad liegen, also zufällig zwischen 0 und 90 Grad. Auch wenn die Verbesserung durch die Minimierung der Bewegung nicht sehr groß ist, so ist sie dennoch relevant, da jede eingesparte Drehung die Druckzeit verringert. Es ist kein anderes Verhalten bei komplexen Bauteilen bemerkbar. Insgesamt, ist die Verbesserung über einer zufälligen Ausrichtung nicht sehr groß. Dennoch ist es eine extreme Verbesserung über der Methode, in dem die Bewegung des Bauteils ignoriert wurde. Die Bewertung der Bewegung eines Bauteils ist immer davon ausgegangen, dass ein Bauteil vor der Ausrichtung immer ohne jegliche vorherige Drehung auf seiner Startposition ist. Eine realistische Bewertung würde den Übergang der letzten zu der aktuellen Fläche messen. Die Flächensuche speichert jedoch nicht, welche Fläche zu welchem Bauteil gehört. Um die Bewegung bei dem Übergang zwischen zwei Flächen zu messen, müsste bekannt sein, aus welchen Flächen ein Bauteil besteht. Dies würde dann eine Druckreihenfolge der Flächen innerhalb eines Bauteils zu bestimmen möglich machen.

Das Finden von Flächen in Bauteilen hat in den meisten Fällen akzeptabel funktioniert. Allerdings besonders bei komplexen Bauteilen, die über viele gebogene Flächen verfügen, hat die Suche häufig Flächen mit Löchern oder anderen Problemen ergeben. Die Löcher in den Flächen sind in sich selber kein Problem. Die Suche hat ein Loch in der Fläche gelassen, weil dort die Punkte einen zu unterschiedlichen Normalvektor hatten. Zwar arbeitet die Flächensuche hier korrekt, denn ein Loch in einer Fläche könnte das Nutzen von Stützstrukturen wieder nötig machen. Durch den Erfolg bei einfachen Bauteilen, lässt sich schlussfolgern, dass die Flächensuche besonders gut darin ist, Flächen die nicht gebogen sind zu finden und die sich durch einen Knick in der Geometrie von anderen Flächen abgrenzen lassen.

Es lässt sich sagen, dass das beschriebene neurale Netz in der Lage ist, die Ausrichtung von Bauteilen zu erkennen. Ob die Abweichung gering genug ist, um praxisrelevant zu sein, ist fragwürdig. Eine geringere Abweichung von dem Optimum wäre wünschenswert, aber die erzielten Ergebnisse sind dennoch ausreichend, um einen Überhang, der Stützstrukturen nötig macht, zu vermeiden. Die Flächensuche funktioniert in den meisten Fällen gut, aber es gibt noch Raum für Verbesserungen, besonders bei komplexen Bauteilen mit gebogenen Flächen. Die Minimierung der Bewegung des Bauteils ist funktional, aber die Verbesserung ist nur gering. Insgesamt ist das beschriebene Verfahren ein akzeptabler Ansatz, um Bauteile in der additiven Fertigung auszurichten, wenn auch noch einige Verbesserungen möglich wären. Es ist für den praktischen Einsatz unter Umständen vorstellbar. Die Aufteilung in Flächen ist problembehaftet und müsste für den praktischen Einsatz stark verbessert werden.

6.2 Ausblick

Diese Arbeit hat gezeigt, dass das Ausrichten von Bauteilen mit neuronalen Netzen möglich ist. Es gibt allerdings noch viele Bereiche in denen Verbesserungen vorgenommen werden können.

Die Genauigkeit des Netzes könnte durch eine Verbesserung der Flächensuche gesteigert werden. Die Flächensuche selber basiert auf einem fehlerbehafteten Ansatz. Für unterschiedliche Arten von Bauteilen müssen die Parameter der Flächensuche manuell angepasst werden, was dem Sinn von automatischer Anpassung widerspricht. Eine Flächensuche, die weniger Fehler macht, würde dem neuronalen Netz bessere Trainingsdaten liefern. Dies würde die Genauigkeit des Netzes steigern. Ein weiteres Problem der Flächensuche ist, dass sie Flächen mit Löchern findet. Das ist ein Problem, was mit dem Clustering von Normalvektoren zusammenhängt und dementsprechend vermutlich nicht behoben werden kann, ohne den Ansatz der Flächensuche zu verändern. Eine Flächensuche, die in der Lage ist, Flächen mit Löchern zu vermeiden, würde das Problem der Stützstrukturen lösen.

Die Architektur des Netzes könnte ebenfalls verändert werden, um die Genauigkeit zu steigern. Es könnten andere Architekturen ausprobiert werden, die eventuell besser für das Problem geeignet sind. Auch eine Veränderung der Hyperparameter des Netzes könnte die Genauigkeit verbessern. Außerdem könnte in zukünftigen Training eines neuronalen Netzes die relative Position der Fläche in einem Bauteil und die Position einer Fläche relativ zu seinen benachbarten Flächen berücksichtigt werden.

Dadurch würde ein besserer Übergang zwischen Flächen ermöglicht werden.

Weitere Arbeiten könnten dementsprechend sich auf bessere Flächensuche und Architektur des Netzes konzentrieren.

Abbildungsverzeichnis

1.1	Beispiel für einen 5d-Drucker	2
2.1	Schematische Darstellung eines 3d-Druckers	6
2.2	Vorschau eines Bauteils mit Stützstrukturen	7
2.3	Darstellung der Unterschiede im Bauteil abhängig von der Höhe der Schichten	9
2.4	Darstellung von durch Formoptimierung aufgeteiltem Bauteil (links) und verbesserte Aufteilung durch Formoptimierung (rechts)	10
2.5	Darstellung der beweglichen Plattform eines 5d-Druckers	12
3.1	Darstellung von Tangenten in einer gekrümmten Fläche	17
3.2	Clustering von verstreuten zweidimensionalen Punkten	17
3.3	Darstellung von Normalvektoren an einem Punkt der Bauteiloberfläche	19
3.4	Beispiel von Hierarchie Clustering als Dendrogramm	20
3.5	Hierarchisches Clustering eines Bauteils nach Normalvektor	22
3.6	Beispielhafte Darstellung von DB-Scan Algorithmus	24
4.1	Funktionsweise eines künstlichen Neurons	29
4.2	Schematische Darstellung von Regressionsnetz	30
4.3	Schematische Darstellung von Merkmalsextraktionschichten	33
4.4	Berechnung von n_1	37
4.5	Berechnung von n_2	38
4.6	Vertauschen von n_1 und n_2 in der Rotationsmatrix	38
4.7	Veränderung der Rotationmatrix durch die Spiegelmatrix	40
5.1	Darstellung aller Typen von Bauteilen im Datensatz	42
5.2	Test mit Noise auf dem Neuralen Netz	45
5.3	Test von schlechter Flächensuche auf dem Neuralen Netz	46
5.4	3d-Modelle der komplexen Bauteile	48

Algorithmenverzeichnis

3.1	DB-Scan Algorithmus	25
3.2	ClusterErweitern Algorithmus	25
4.1	Farthest Point Sampling Algorithmus	34

Literatur

- [DE84] William HE Day und Herbert Edelsbrunner. „Efficient algorithms for agglomerative hierarchical clustering methods“. In: *Journal of classification* 1.1 (1984), S. 7–24.
- [Eld+97] Yuval Eldar, Michael Lindenbaum, Moshe Porat und Yehoshua Y Zeevi. „The farthest point strategy for progressive image sampling“. In: *IEEE transactions on image processing* 6.9 (1997), S. 1305–1315.
- [Est+96] Martin Ester, Hans-Peter Kriegel, Jörg Sander, Xiaowei Xu u. a. „A density-based algorithm for discovering clusters in large spatial databases with noise“. In: *kdd*. Bd. 96. 34. 1996, S. 226–231.
- [Fas12] Petra Fastermann. *3d-druck/rapid prototyping: Eine zukunftsstechnologie-kompakt erklärt*. Springer-Verlag, 2012.
- [Fen+18] Jiawei Feng, Jianzhong Fu, Zhiwei Lin, Ce Shang und Bin Li. „A review of the design methods of complex topology structures for 3D printing“. In: *Visual computing for industry, biomedicine, and art* 1.1 (2018), S. 5.
- [GD13] Ade Gunawan und M De Berg. „A faster algorithm for DBSCAN“. In: *Master’s thesis* (2013).
- [Hal+24] James Haley, Jaydeep Karandikar, Callan Herberger, Eric MacDonald, Thomas Feldhausen und Yousub Lee. „Review of in situ process monitoring for metal hybrid directed energy deposition“. In: *Journal of Manufacturing Processes* 109 (2024), S. 128–139.
- [Jia+18] Jingchao Jiang, Jonathan Stringer, Xun Xu und Ray Y Zhong. „Investigation of printable threshold overhang angle in extrusion-based additive manufacturing for reducing support waste“. In: *International Journal of Computer Integrated Manufacturing* 31.10 (2018), S. 961–969.
- [KM21] Nikhil Ketkar und Jojo Moolayil. „Convolutional neural networks“. In: *Deep learning with Python: learn best practices of deep learning models with PyTorch*. Springer, 2021, S. 197–242.

- [Kum+26] Mohit Kumar, Jarnail Singh, Gaurav Arora, Ranvijay Kumar, Sanjay Mavinkere Rangappa und Suchart Siengchin. „Recent Progress in 5D Printing: Processes, Materials, Applications, and Future Trends“. In: *Advanced Materials Technologies* (2026), e01337.
- [OP16] William Oropallo und Les A Piegl. „Ten challenges in 3D printing“. In: *Engineering with Computers* 32.1 (2016), S. 135–148.
- [Qi+17a] Charles R Qi, Hao Su, Kaichun Mo und Leonidas J Guibas. „Pointnet: Deep learning on point sets for 3d classification and segmentation“. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2017, S. 652–660.
- [Qi+17b] Charles Ruizhongtai Qi, Li Yi, Hao Su und Leonidas J Guibas. „PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space“. In: *Advances in Neural Information Processing Systems*. Hrsg. von I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan und R. Garnett. Bd. 30. Curran Associates, Inc., 2017.
- [RD18] P Ravinder Reddy und P Anjani Devi. „Review on the advancements of additive manufacturing-4D and 5D printing“. In: *Int J Mech Prod Eng Res Dev* 8.4 (2018), S. 397–402.
- [SLR19] Nurhalida Shahrubudin, Te Chuan Lee und RJPM Ramlan. „An overview on 3D printing technology: Technological, materials, and applications“. In: *Procedia manufacturing* 35 (2019), S. 1286–1296.
- [SM13] Ole Sigmund und Kurt Maute. „Topology optimization approaches: A comparative review“. In: *Structural and multidisciplinary optimization* 48.6 (2013), S. 1031–1055.
- [Sta15] IOf Standardization. „Additive manufacturing: general: principles“. In: *Terminology* 640 (2015).
- [Wan03] Sun-Chong Wang. „Artificial Neural Network“. In: *Interdisciplinary Computing in Java Programming*. Boston, MA: Springer US, 2003, S. 81–100.
- [War63] Joe H Ward Jr. „Hierarchical grouping to optimize an objective function“. In: *Journal of the American statistical association* 58.301 (1963), S. 236–244.

- [WW25] Inc Wohlers Associates und T.T. Wohlers. *Wohlers Report 2025: 3D Printing and Additive Manufacturing : Global State of the Industry*. Wohlers Associates, 2025.
- [Zel+25] Sebastian Zelder, Iryna Shevchenko, Joschka zur Jacobsmühlen, Jörg Krüger und Eckart Uhlmann. „Learning automatic part orientation for powder bed fusion of metal from part data“. In: *Progress in Additive Manufacturing* 10.3 (2025), S. 1713–1719.
- [Zhu+24] Zicheng Zhuang, Fengming Xu, Junhong Ye, Nan Hu, Liming Jiang und Yiwei Weng. „A comprehensive review of sustainable materials and toolpath optimization in 3D concrete printing“. In: *npj Materials Sustainability* 2.1 (2024), S. 12.

Erklärung zur Verwendung von Hilfsmitteln und KI-gestützten Schreibwerkzeugen

Ich versichere, dass ich beim Einsatz von IT-/KI-gestützten Schreibwerkzeugen diese Werkzeuge in der nachfolgenden Übersicht der verwendeten Hilfsmittel mit ihrem Produktnamen und meiner Bezugsquelle vollständig aufgeführt und/oder die betreffenden Textstellen in der Arbeit als mit IT/KI generierter Unterstützung verfasst gekennzeichnet habe.

Mir ist bewusst, dass Täuschungen bzw. Täuschungsversuche nach der für mich geltenden Prüfungsordnung geahndet werden.

Folgende Hilfsmittel und KI-gestützten Schreibwerkzeuge habe ich für die Bearbeitung genutzt:

- LanguageTool, LanguageTool, 1.0.1, Rechtschreibung und Grammatik, <https://languagetool.org/de>
- ChatGPT, ChatGPT, 5, LaTeX Syntax Fehlerbehebung, <https://chatgpt.com/>